

CS2 AI COACHING SYSTEM

Ultimate CS2 Coach

Parte 1A — O Cerebro

Arquitetura neural, treinamento, JEPA, VL-JEPA, Superposition Layer, LSTM + Mixture of Experts

Autore: Renan Augusto Macena

Marzo 2026

Ultimate CS2 Coach - Parte 1A: O Cerebro

Topicos: Arquitetura neural completa, regime de treinamento, modelos de deep learning (JEPA, VL-JEPA, LSTM+MoE), Observatorio de Introspeccao do Coach, contrato 25-dim, principio NO-WALLHACK, e panorama da arquitetura de sistema (inicializacao, interface desktop Qt/PySide6, arquitetura quad-daemon, pipeline geral).

Autor: Renan Augusto Macena

Introducao de Renan

Este projeto e o resultado de incontaveis horas de preparacao, aperfeicoamento e, sobretudo, pesquisa. Counter-Strike e o meu futebol, algo pelo qual sou apaixonado a vida toda, como a musica. Neste jogo eu ja tenho mais de 10 mil horas e jogo desde 2004, sempre desejei uma orientacao profissional, como a dos jogadores profissionais de verdade, para entender como realmente parece quando alguem treina do jeito certo, e joga do jeito certo sabendo o que e certo ou errado e nao apenas supondo. Neste ponto a minha confianca esta ligada ao meu conhecimento do jogo, mas ainda esta longe do nivel em que eu gostaria de jogar. Imagino que muitas pessoas como eu, neste jogo ou talvez em outros, se sintam do mesmo jeito: sabem que tem experiencia e conhecimento do que estao fazendo, mas percebem que os jogadores profissionais sao tao mais habilidosos e refinados que nem mesmo anos de experiencia podem igualar uma fracao do que esses profissionais oferecem.

Este projeto tenta "dar vida", usando tecnicas avancadas, a um coach definitivo para Counter-Strike, que entenda o jogo, avalie dezenas de aspectos com clareza e julgamento refinado, assimile e se torne mais inteligente com o passar do tempo.. Meu objetivo final e faze-lo ingerir, processar e aprender de todas as partidas pro de sempre, nao todos os playoffs, alem de ser irrealista pelo tamanho e tempo de processamento, seria inutil pois o meu objetivo e ter este coach definitivo baseado no melhor do melhor. O modelo para o jogador "de baixa habilidade" sera sempre o usuario, ele ou ela nunca tera demos e partidas das quais este coach possa aprender; em vez disso, este e um metodo de abordagem completamente diferente, ja que o coach utilizara seu modelo de alta qualidade para compara-lo com o modelo do usuario, mostrando o quanto o usuario esta realmente distante do nivel de um jogador profissional e, mais importante, adaptando os ensinamentos para ajudar o usuario a alcançar o nivel pro, adaptando-se a cada estilo diferente de jogo: se eu sou um awper, lentamente mas inexoravelmente este coach me ensinara como me tornar um awper profissional, e isso vale para cada papel no jogo em que um usuario queira se tornar um profissional.

E agora com a paixao que estou desenvolvendo pela programacao, entendendo toda essa coisa complicada sobre gerenciamento de banco de dados, SQL, modelos de machine learning, Python, e o quao elegante pode ser, de todos esses passos tecnicos de implementacao, ajuste e criacao de APIs, e entendendo o que e aquela .dll que eu vi a vida toda dentro das pastas do jogo, entendendo para que servem aqueles frameworks como Kivy para criar a parte grafica, aprendendo o que realmente significa criar software, tive que aprender a ir para o Linux (definitivamente nao sou um profissional, e se eu nao tivesse seguido as mesmas instrucoes que criei enquanto pesquisava e continuava trabalhando neste projeto, nao teria conseguido) para entender como construir o programa, como torna-lo cross-platform. Do Linux, construi a versao APK (tenho que terminar de trabalhar nela tambem), e pelo menos entendi os conceitos mais importantes da Compilacao, depois terminei a build desktop no Windows e fiz todo o resto, compilacao e empacotamento. Em resumo, se empurrar ao limite para entender, desafiar-se tanto, em tantos aspectos diferentes que voce precisa ou assimilar nao apenas algo mas tambem as especificidades e o quadro complexo de tudo, ou nao chegara a lugar nenhum.

Usei ferramentas, como Claude CLI no terminal windows e linux, para me ajudar a organizar o codigo e corrigir os erros de sintaxe. Criei uma pasta na pasta do projeto contendo uma "Tool Suite" composta por scripts Python que ajudei a criar e que tem uma tonelada de funcoes uteis, do debug a modificacao de valores, regras, funcoes, strings em nivel global ou local do projeto de modo que a partir de uma unica entrada dentro daquela ferramenta, eu possa mudar tudo o que quiser "em qualquer lugar", ate mesmo mudar coisas no dashboard grafico com os inputs. Nao durmo ha cerca de vinte dias consecutivos (desde 24 de dezembro de 2025), sim, mas acho que vale a pena, e sei bem que este e apenas um treinamento no final que eu entendi

como fazer sozinho do inicio ao fim, entao certamente ha erros por ai. Se pode ser realmente util, realmente funcional, etc., etc., e muitas outras coisas bonitas, nao sei. Nao quero superestimar o que estou fazendo. Estou fazendo, mas coloquei muito esforco mesmo.

Espero que algo ali dentro possa ser util.

Nota: O framework UI foi posteriormente migrado de Kivy para Qt/PySide6 (marco de 2026), como documentado na Parte 3.

Indice

Parte 1A - O Cerebro: Arquitetura Neural e Treinamento (este documento)

1. [Resumo executivo](#)
2. [Panorama da arquitetura de sistema](#)
 - Principio NO-WALLHACK e Contrato 25-dim
3. [Subsistema 1 - Nucleo da rede neural \(backend/nn/ \)](#)
 - AdvancedCoachNN (LSTM + MoE)
 - JEPA (Auto-Supervisionado InfoNCE)
 - **VL-JEPA** (Visao-Linguagem, 16 Conceitos de Coaching, ConceptLabeler)
 - JEPATrainer (Treinamento + Monitoramento Drift)
 - Pipeline Standalone (jepa_train.py)
 - SuperpositionLayer (Gating Contextual, Observabilidade Avancada, Inicializacao Kaiming)
 - Modulo EMA
 - CoachTrainingManager, TrainingOrchestrator, ModelFactory, Config
 - NeuralRoleHead (Classificacao de Papeis MLP)
 - Coach Introspection Observatory (MaturityObservatory - Maquina de 5 Estados)

Parte 1B - Os Sentidos e o Especialista: Modelo RAP Coach (arquitetura 7 componentes, ChronovisorScanner, GhostEngine), Fontes de Dados (Demo Parser, HLTV, Steam, FACEIT, TensorFactory, FAISS)

Parte 2 - Secoes 5-13: Servicos de Coaching, Coaching Engines, Conhecimento e Recuperacao, Motores de Analise (11), Processamento e Feature Engineering, Modulo de Controle, Progresso e Tendencias, Banco de Dados e Storage (Tri-Tier), Pipeline de Treinamento e Orquestracao, Funcoes de Perda

Parte 3 - Logica do Programa, UI, Ingestao, Tools, Tests, Build, Remediacao

1. Resumo executivo

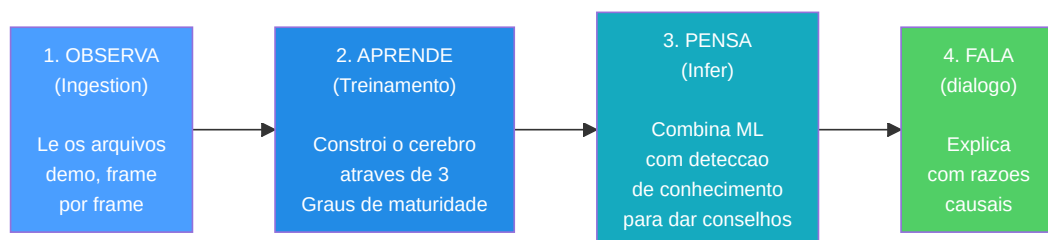
CS2 Ultimate e um **sistema de coaching baseado em IA hibrido** para Counter-Strike 2 (CS2). Combina modelos de deep learning (JEPA, VL-JEPA, LSTM+MoE, uma arquitetura RAP de 7 componentes (Percepcao, Memoria LTC+Hopfield, Estrategia, Pedagogia, Atribuicao Causal, Cabeça de Posicionamento + Comunicacao externa), um Neural Role Classification Head), um Coach Introspection Observatory, um Retrieval-Augmented Generation (RAG), o banco de experiencias COPER, a busca baseada em teoria dos jogos, a modelagem bayesiana das crenças e uma arquitetura Quad-Daemon (Hunter, Digester, Teacher, Pulse) em uma pipeline unificada que:

1. **Ingere** arquivos demo profissionais e de usuario, extraindo estatisticas de estado em nivel de tick e de partida.
2. **Treina** varios modelos de redes neurais atraves de um programa em fases com limites de maturidade (em 3 niveis: CALIBRACAO -> APRENDIZADO -> MADURO).
3. **Infere** conselhos de treinamento fundindo as previsoes de machine

learning com conhecimentos taticos recuperados semanticamente atraves de uma cadeia de fallback de 4 niveis (COPER -> Híbrido -> RAG -> Base).

3. **Explica** seu raciocinio atraves de atribuicao causal, narrativas baseadas em template, comparacoes entre jogadores profissionais e um refinamento LLM opcional (Ollama).

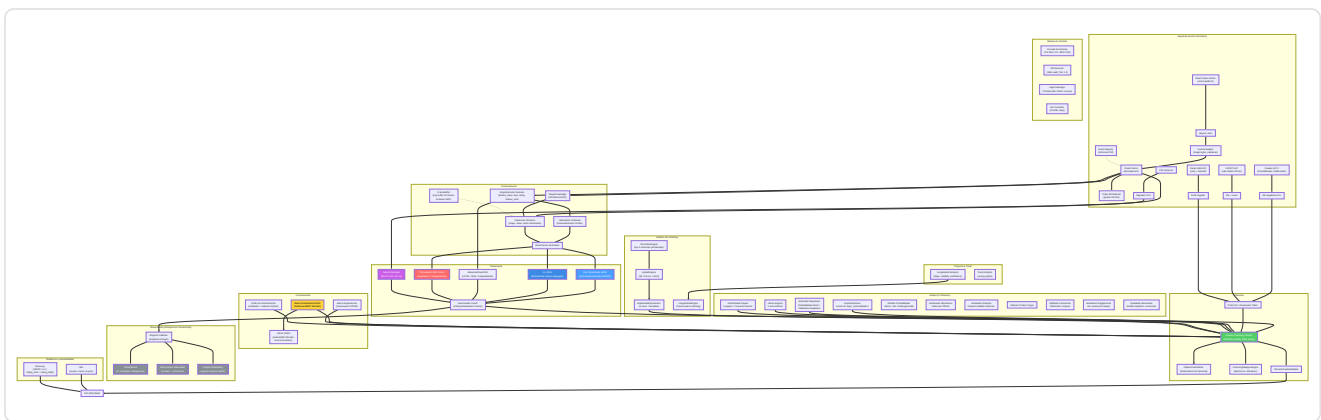
O sistema contem ~**103.600 linhas de Python** distribuidas em 445 arquivos `.py` sob `Programma_CS2_RENAN/`, que se estendem por **oito subsistemas logicos de IA** (NN Core com VL-JEPA, RAP Coach + RAP Lite, Coaching Services, Knowledge & Retrieval, Analysis Engines (11), Processing & Feature Engineering, Fontes de Dados, Motores de Coaching), um Observatorio de treinamento, um modulo de Controle (Console com REST API, DB Governor, Ingest Manager, ML Controller), uma arquitetura Quad-Daemon para automacao em background (Hunter, Digester, Teacher, Pulse), uma interface desktop Qt/PySide6 com 15 telas e pattern MVVM (migrada de Kivy em marco de 2026), um sistema completo de ingestao (com subsistema HLTV dedicado: HLTVApiService, CircuitBreaker, RateLimiter), storage e reporting, uma **Tools Suite** com 41 scripts Python de validacao e diagnostico (29 root + 12 no pacote - Goliath Hospital, headless validator, Ultimate ML Coach Debugger, `validate_coaching_pipeline`, `ingest_pro_demos`, `dead_code_detector`, `dev_health`, `rebuild_monolith`, `tick_census`), uma arquitetura tri-database especializada com 18 tabelas SQLAlchemy no monolito (+ 3 no banco HLTV separado + 3 nos bancos por-match = 24 totais), e uma **Test Suite** com 130 arquivos de teste organizados em 6 categorias: analysis/theory, coaching/training, ML/models, data/storage, UI/playback, integration/misc. O projeto passou por um processo de **remediacao sistematica em 13 fases** que resolveu 412+ problemas de qualidade do codigo, correcao ML, seguranca e arquitetura, incluindo a eliminacao de label leakage no treinamento (G-01), a implementacao da zona de perigo visual no tensor de visao (G-02), a calibracao automatica do estimador bayesiano (G-07), e a correcao do fallback do coaching COPER (G-08), seguida por uma **segunda onda de remediacao** que resolveu mais 162 problemas (31 HIGH + 131 MEDIUM) relacionados a thread safety, schema drift, Qt lifecycle, e hardening de observabilidade, e uma **terceira onda** (abril de 2026) que enderecou 40+ problemas adicionais incluindo type safety (SA-14 a SA-27), dependency pinning (DEP-1), checkpoint security (CTF-1/2), DataLineage audit trail (DL-1), e correcoes UI/UX (UX-1/2/3). A pipeline end-to-end foi completada em 12 de marco de 2026: 11 demos profissionais ingeridas, 17.3M linhas de tick, 6.4GB de banco de dados, JEPA pre-treinado (train loss 0.9506, val loss 1.8248). Atualizacao abril 2026: 156 bancos de dados por-match, AdvancedCoachNN completamente treinado (`latest.pt`), banco HLTV populado com **161 jogadores profissionais reais** (32 times, 156 stat cards), HybridCoachingEngine potencializado com selecao automatica do pro de referencia por nome nos feedbacks de coaching, **Coach Book v4** expandido para **502 entradas** de conhecimento tatico em 8 arquivos JSON (7 mapas + geral) com 13 categorias, **CoachingDialogueEngine** para coaching multi-turno com drill-down por-jogador e por-round, **MovementQualityAnalyzer** (11o motor de analise), **EloAugmentedPredictor** para probabilidade de vitoria com feature Elo, **PlusMinus rating metric**, potencializacoes COPER (incerteza TrueSkill, semantica CRUD, replay priorizado), codificacao posicao relativa ao bombsite, priorizacao de demos por variancia de coaching com scoring de qualidade via modelo Huber, e **CS2 Coach Bench** benchmark de avaliacao com 200 perguntas.



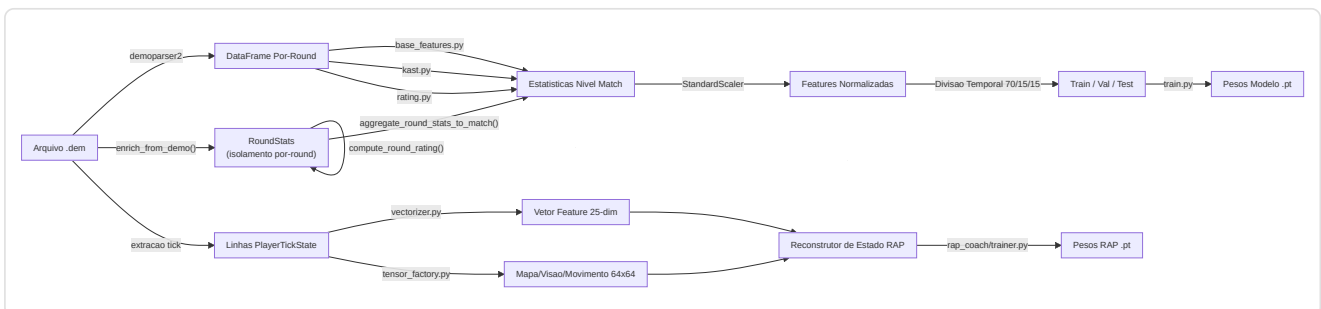
445 .py files - 102.000+ lines - 8 AI subsystems + Observatory + Control Module (REST API) + Quad-Daemon + Desktop UI Qt/PySide6 (15 screens) + 41 Tools (29 root + 12 inner) - 130 test files - 21 tabelas SQLModel (monolito) + 3 per-match - Arquitetura tri-database (database.db + hltv_metadata.db + banco per-match) - Indexacao vetorial FAISS (IndexFlatIP 384-dim) - Internacionalizacao i18n (EN/IT/PT) - Acessibilidade WCAG 1.4.1 (theme.py) - 12 relatorios de auditoria abrangentes (incl. revisao de literatura 140KB, 30 artigos peer-reviewed) - 610+ problemas resolvidos (412 em 13 fases + 162 em segunda onda + 40+ em terceira onda) - Pipeline end-to-end completada (156 per-match DB - JEPA + AdvancedCoachNN treinados - 161 jogadores HLTV reais, 32 times, 156 stat cards - Coach Book v4: 502 entradas, 8 arquivos, 13 categorias - CS2 Coach Bench: 200 perguntas)

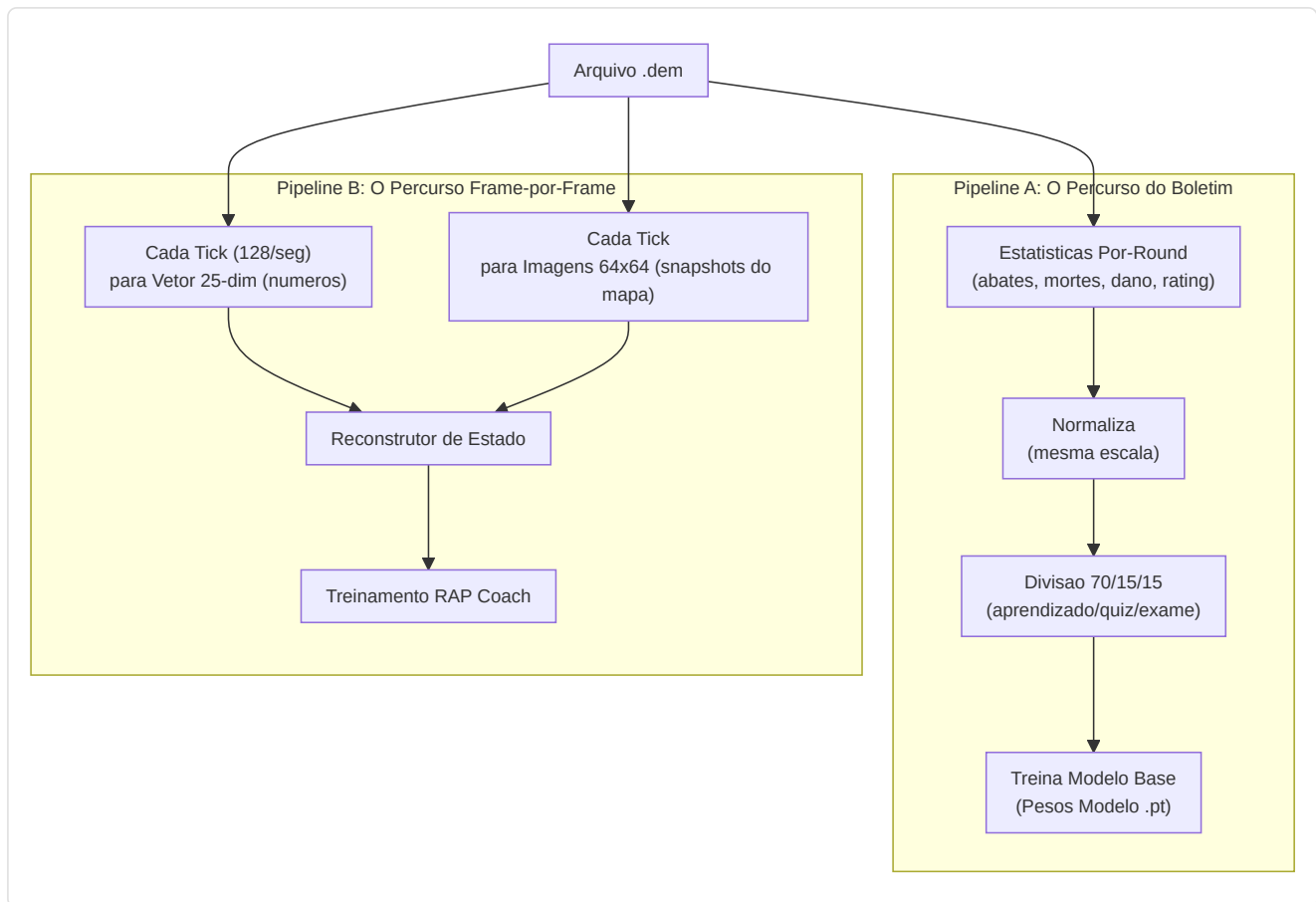
2. Panorama da arquitetura do sistema

O sistema e dividido em **6 subsistemas principais** com responsabilidades distintas. Cada subsistema tem uma tarefa especifica e os dados fluem entre eles em uma pipeline bem definida.



Resumo do fluxo de dados





Princípio NO-WALLHACK e Contrato 25-dim

Dois invariantes arquiteturais fundamentais atravessam o sistema inteiro:

- 1. Princípio NO-WALLHACK:** O coach IA **ve apenas o que o jogador legitimamente conhece**. Quando o módulo **PlayerKnowledge** está disponível, os tensores gerados pela **TensorFactory** codificam exclusivamente informações legítimas: companheiros de equipe (sempre visíveis), inimigos em posições "last-known" (com decaimento temporal, $\tau = 2.5s$), utilidade própria e observada. Nenhuma informação "wallhack" (posições inimigas reais não visíveis) entra jamais no sistema de percepção. Quando **PlayerKnowledge** é **None**, o sistema recai em um modo legacy com tensores simplificados.
- 2. Contrato 25-dim (FeatureExtractor):** O **FeatureExtractor** em **vectorizer.py** define o vetor de feature canônico de 25 dimensões (**METADATA_DIM = 25**) usado por **todos** os modelos (AdvancedCoachNN, JEPA, VL-JEPA, RAP Coach) tanto em treinamento quanto em inferência. Qualquer modificação no vetor de features ocorre **exclusivamente** no **FeatureExtractor** - nenhum outro módulo pode definir features próprias. Isso garante coerência dimensional end-to-end.

```

0: health/100      1: armor/100      2: has_helmet      3: has_defuser
4: equip/10000     5: is_crouching   6: is_scoped       7: is_blinded
8: enemies_vis     9: pos_x/4096     10: pos_y/4096     11: pos_z/1024
12: view_x_sin     13: view_x_cos    14: view_y/90      15: z_penalty
16: kast_est       17: map_id        18: round_phase
19: weapon_class   20: time_in_round/115 21: bomb_planted
22: teammates_alive/4 23: enemies_alive/5 24: team_economy/16000

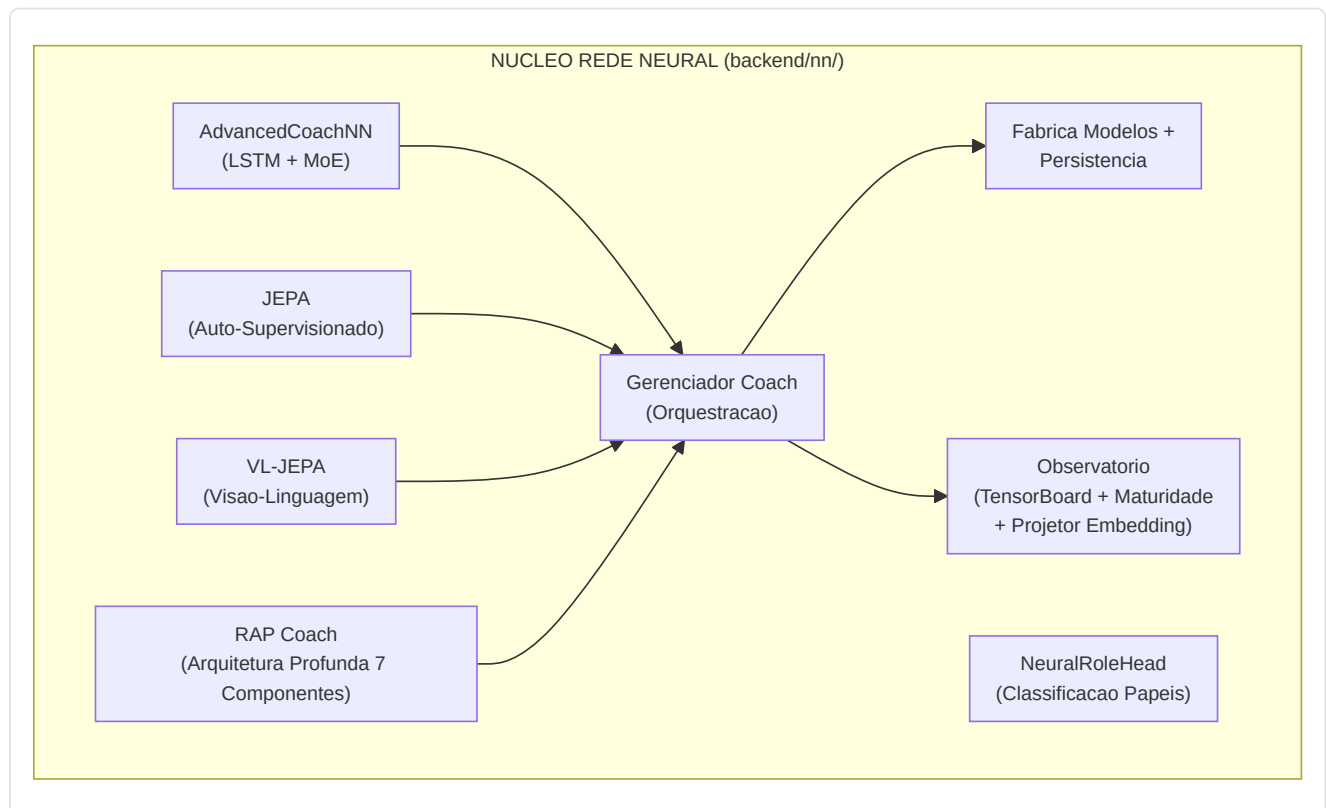
```

Normalização posição: ``np.clip(pos_x / cfg.pos_xy_extent, -1.0, 1.0)`` onde ``pos_xy_extent=4096.0`` (default, configurável por mapa). O clip em `[-1,1]` protege de coordenadas fora do intervalo que produziriam features `> 1.0` em módulos não normalizados.

3. Subsistema 1 - Nucleo da rede neural

Pasta no programa: `backend/nn/` **Arquivos-chave:** `model.py`, `jepa_model.py`, `jepa_train.py`, `jepa_trainer.py`, `coach_manager.py`, `training_orchestrator.py`, `config.py`, `factory.py`, `persistence.py`, `role_head.py`, `training_callbacks.py`, `tensorboard_callback.py`, `maturity_observatory.py`, `embedding_projector.py`, `dataset.py`, `data_quality.py`, `evaluate.py`, `train_pipeline.py`, `training_monitor.py`, `early_stopping.py`, `ema.py`, `training_controller.py`, `win_probability_trainer.py`, `training_config.py`

Este subsistema contém todos os modelos de rede neural, o "cerebro" do sistema de coaching. Inclui seis arquiteturas distintas de modelos (AdvancedCoachNN, JEPA, VL-JEPA, RAP Coach, RAP Lite, NeuralRoleHead), um gerenciador de training, um Observatorio de Introspeccao do Coach e utilitarios para a criacao e persistencia dos modelos.



-AdvancedCoachNN (LSTM + Mistura de Especialistas)

Definido em `model.py`, este é o fundamento do coaching supervisionado.

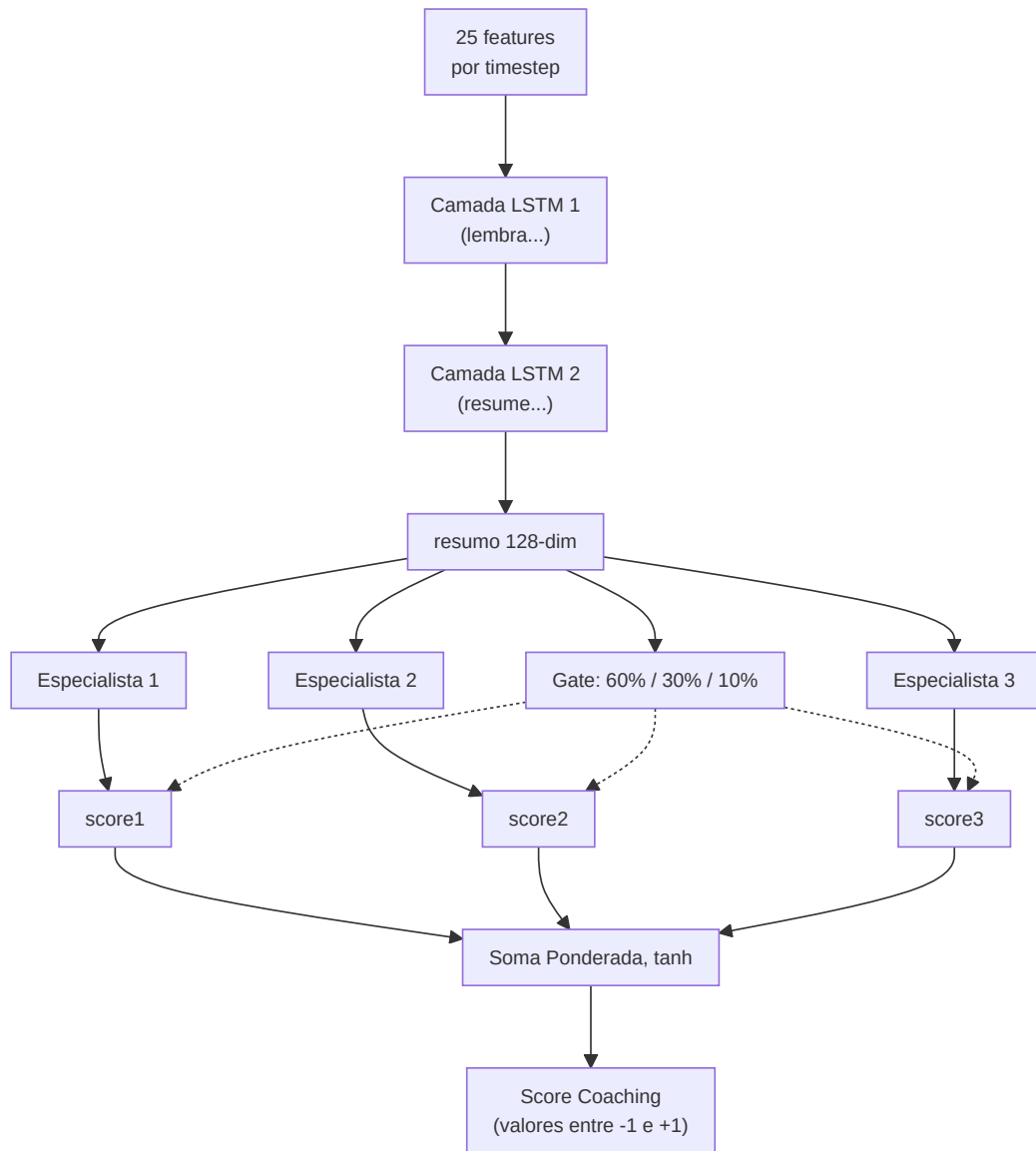
Componente	Detalhe
Dimensao de input	25 funcionalidades (<code>METADATA_DIM</code> de <code>vectorizer.py</code>)
Config	Dataclass <code>CoachNNConfig</code> : <code>input_dim=25</code> , <code>output_dim=METADATA_DIM</code> (default 25, mas sobrescrito para <code>OUTPUT_DIM=10</code> pela <code>ModelFactory</code>), <code>hidden_dim=128</code> , <code>num_experts=3</code> , <code>num_lstm_layers=2</code> , <code>dropout=0.2</code> , <code>use_layer_norm=True</code>
Camadas ocultas	LSTM de 2 camadas (128 ocultas, <code>batch_first=True</code> , <code>dropout=0.2</code>) com <code>LayerNorm</code> pos-LSTM
Cabeca dos especialistas	3 especialistas lineares paralelos (configuraveis), softmax-gated atraves de uma rede de gate aprendida
Output	Soma ponderada dos outputs dos especialistas -> vetor do score de coaching de 10 dimensoes. A <code>CoachNNConfig</code> define <code>output_dim=METADATA_DIM</code> (25) como default, mas a <code>ModelFactory</code> sobrescreve com <code>OUTPUT_DIM=10</code> em producao. O alias <code>TeacherRefinementNN = AdvancedCoachNN</code> e mantido para retrocompatibilidade
Bias de papel	Parametro <code>role_id</code> opcional: <code>gate_weights = (gate_weights + role_bias) / 2.0</code> - orienta a selecao dos especialistas para conhecimentos especificos do papel
Validacao do input	<code>_validate_input_dim()</code> redimensiona automaticamente 1D -> <code>unsqueeze(0).unsqueeze(0)</code> e 2D -> <code>unsqueeze(0)</code> para robustez

Cada modulo especialista em `AdvancedCoachNN`: `Linear(128->128)` -> `LayerNorm(128)` -> `ReLU` -> `Linear(128->output_dim)`.

Nota: `_create_expert()` do JEPA omite `LayerNorm` - apenas `Linear -> ReLU -> Linear`. Trata-se de uma escolha de design deliberada: os especialistas JEPA operam em embeddings latentes ja normalizados, enquanto os especialistas `AdvancedCoachNN` processam outputs LSTM brutos que se beneficiam da normalizacao por especialista.

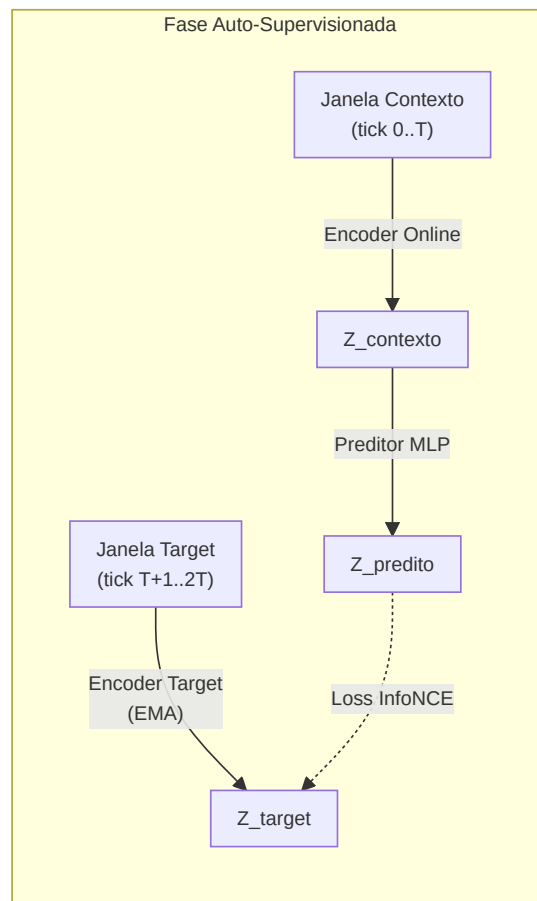
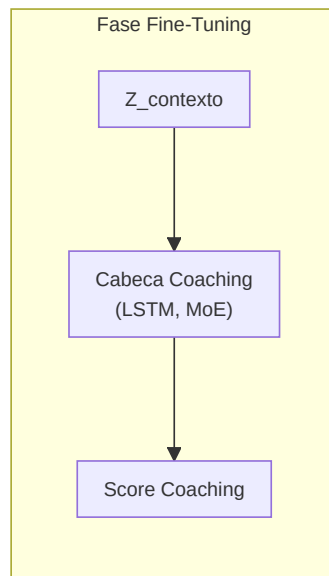
Passagem em avanco (pseudo forward pass):

```
h, _ = LSTM(x) # x: [batch, seq_len, 25]
h = LayerNorm(h[:, -1, :]) # pega o ultimo timestep -> [batch, 128]
gate_weights = softmax(W_gate . h) # [batch, 3]
expert_outputs = [E_i(h) for i in 1..3]
output = tanh(Sigma gate_weights_i x expert_outputs_i)
```



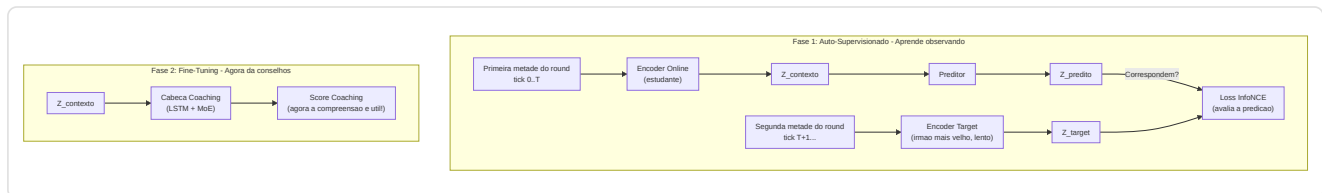
-Modelo de coaching JEPA (Arquitetura preditiva com integracao conjunta)

Definido em `jepa_model.py`. Um modelo de **pre-treinamento auto-supervisionado** inspirado no I-JEPA de Yann LeCun, adaptado para dados CS2 sequenciais.



Detalhes da arquitetura:

Modulo	Parametros
Codificador online	Linear(input_dim, 512) -> LayerNorm -> GELU -> Dropout(0.1) -> Linear(512, latent_dim=256) -> LayerNorm
Codificador target	Estruturalmente identico; atualizado via media movel exponencial (tau = 0.996). <code>EMA.state_dict()</code> retorna tensores clonados para prevenir aliasing (um bug anterior permitia a modificacao acidental dos pesos target atraves de referencias compartilhadas)
Predictor	Linear(256, 512) -> LayerNorm -> GELU -> Dropout(0.1) -> Linear(512, 256)
Coaching Head	LSTM(256, hidden_dim, 2 layers, dropout=0.2) -> 3 especialistas MoE -> output controlado



Procedimento de pre-treinamento (`jepa_trainer.py`):

- Carrega as sequencias de `PlayerTickState` dos arquivos SQLite de demos profissionais.
- Divide cada sequencia em janelas de contexto e target.
- Codifica o contexto atraves do codificador online + preditor, codifica o target atraves do codificador do target (EMA).
- Minimiza a **perda de contraste de InfoNCE** utilizando negativos em batch com similaridade do cosseno e temperatura $\tau=0,07$.
- Depois de cada batch, executa o update EMA: `theta_target <- tau.theta_target + (1-tau).theta_online`.
- Monitoramento do drift:** Rastreia os objetos `DriftReport`; ativa o retreinamento automatico se o drift > 2,5 sigma.
- Rotulos baseados em resultado (Correcao G-01):** O `ConceptLabeler` no treinamento VL-JEPA agora gera rotulos a partir dos dados `RoundStats` (resultados por round: abates, mortes, danos, sobrevivencia) em vez de features em nivel de tick. Isso elimina o **label leakage** - o problema anterior em que os rotulos dos conceitos eram derivados das mesmas features usadas como input, permitindo que o modelo "trapaceasse" durante o treinamento sem realmente aprender os patterns. O metodo `label_from_round_stats(rs)` produz um vetor de 16 rotulos de conceito baseados em resultados mensuraveis. Se os dados `RoundStats` nao estiverem disponiveis, o sistema recai na heuristica legacy com um aviso de log uma unica vez.

Decodificacao Seletiva (`forward_selective`): pula a passagem em avanco inteira se a distancia do cosseno entre o embedding atual e o anterior for inferior a um limite (`skip_threshold=0.05`). Utiliza `1.0 - F.cosine_similarity()` como metrica de distancia e, durante a operacao de salto, retorna o output anterior memorizado na cache. Isso permite uma inferencia eficaz em tempo real com salto dinamico dos frames: durante os momentos de jogo estaticos (os jogadores mantem os angulos), a maioria dos frames e pulada completamente.

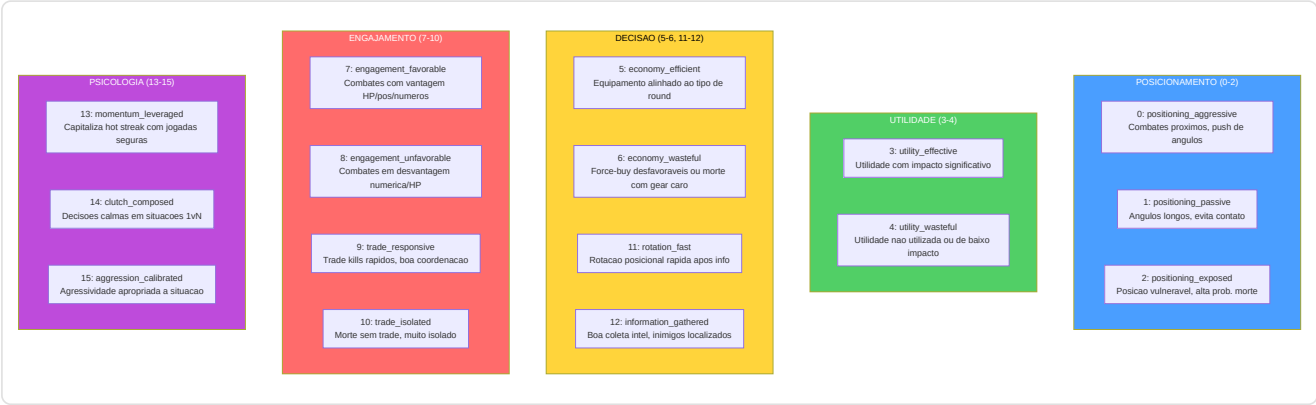
Adaptador JEPA para Coaching (`jepa_insight_adapter.py`): converte os vetores de saida brutos da Coaching Head em objetos `InsightCandidate` prontos para a pipeline de coaching. Mapeia os primeiros 10 elementos do contrato 25-dim em pares de mensagem aumento/diminuicao por eixo tatico (sobrevivencia, economia, movimento, utilidade, posicionamento). A saida sigmoid e normalizada em deltas com sinal `2*(out-0.5)`; deltas com magnitude abaixo de 0.1 sao descartados como ruído. O numero de candidatos e o multiplicador de confianca sao escalados pelo nivel de maturidade: `doubt/crisis` → 0,5×, 1 candidato; `learning` → 0,8×, 2 candidatos; `conviction/mature` → 1,0×, 3 candidatos. Ativado pela flag `USE_JEPA_MODEL` (desligada por padrao, F1.2 — comportamento byte-identico ao atual quando desligada). Respeita a restricao NO-WALLHACK: consome exclusivamente a janela de ticks do jogador observado (contrato POV 25-dim, P-X-01).

-VL-JEPA: Arquitetura de Alinhamento Visao-Linguagem com Conceitos de Coaching

Definido na segunda metade de `jepa_model.py`. O VL-JEPA (Vision-Language JEPA) e uma **extensao fundamental** do JEPACoachingModel que adiciona um **mecanismo de alinhamento entre embeddings latentes e conceitos de coaching interpretaveis**. Inspirado no VL-JEPA de Meta FAIR (2026), mapeia as representacoes latentes em um espaco conceitual estruturado com 16 conceitos de coaching predefinidos.

Taxonomia dos 16 Conceitos de Coaching

O sistema define `NUM_COACHING_CONCEPTS = 16` conceitos organizados em **5 dimensoes taticas**:



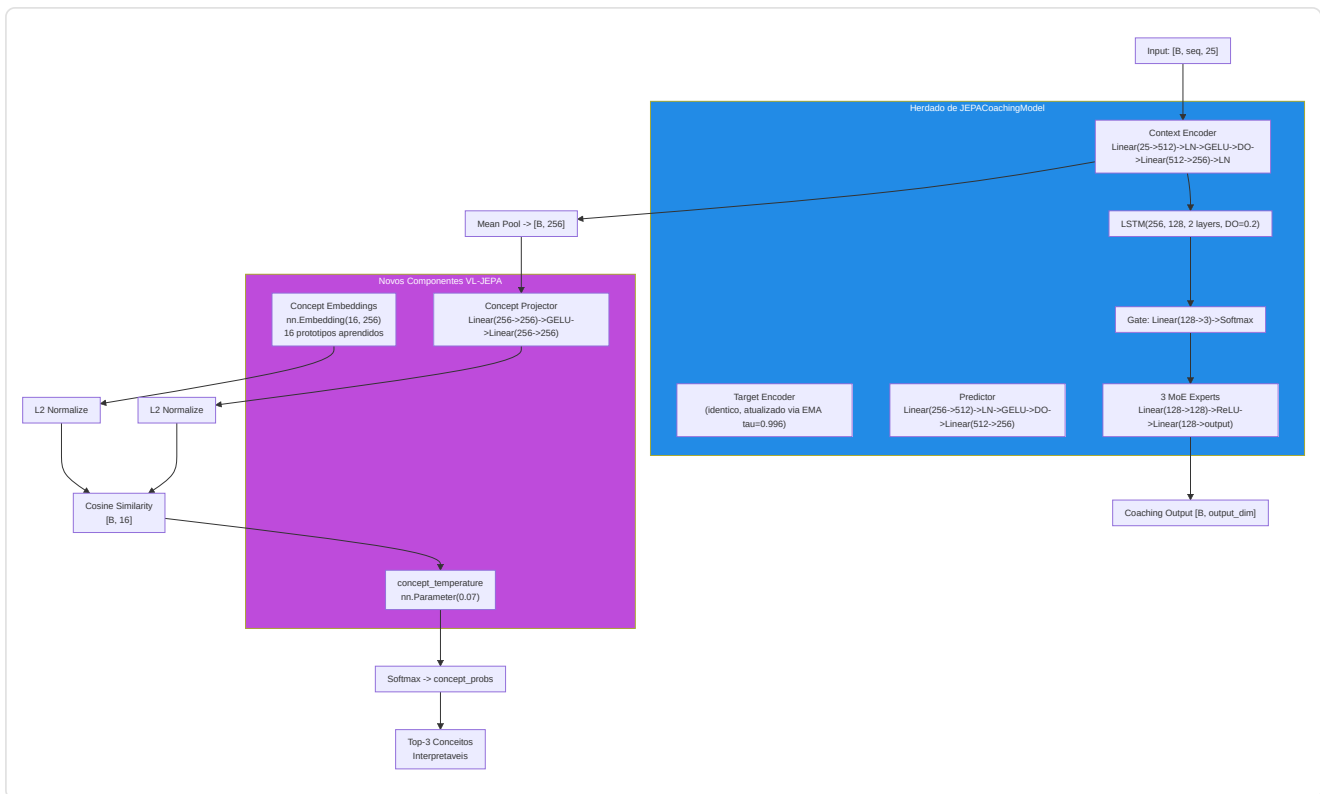
Cada conceito e definido como um `CoachingConcept` dataclass imutavel com `(id, name, dimension, description)`. A lista global `COACHING_CONCEPTS` e `CONCEPT_NAMES` sao as fontes de verdade para todo o sistema.

Arquitetura VLJEPACoachingModel

`VLJEPACoachingModel` herda de `JEPACoachingModel` e adiciona 3 componentes:

Componente	Parametros	Proposito
<code>concept_embeddings</code>	<code>nn.Embedding(16, latent_dim=256)</code>	16 prototipos de conceito aprendidos no espaco latente
<code>concept_projector</code>	<code>Linear(256->256) -> GELU -> Linear(256->256)</code>	Projeta embeddings encoder no espaco alinhado aos conceitos
<code>concept_temperature</code>	<code>nn.Parameter(0.07)</code> , clamped <code>[0.01, 1.0]</code>	Temperatura aprendida para scaling da similaridade cos seno

Todos os caminhos forward do pai (`forward`, `forward_coaching`, `forward_selective`, `forward_jepa_pretrain`) sao **preservados intactos** via heranca. O novo caminho `forward_vl()` adiciona o alinhamento conceitual.



Caminho Forward VL-JEPA (`forward_vl`)

```

1. Encode:     embeddings = context_encoder(x)                # [B, seq, 256]
2. Pool:       latent = embeddings.mean(dim=1)                # [B, 256]
3. Project:    projected = L2_normalize(concept_projector(latent)) # [B, 256]
4. Similarity: logits = projected @ concept_embs_norm.T      # [B, 16]
5. Temperature: probs = softmax(logits / clamp(temp, 0.01, 1.0)) # [B, 16]
6. Coaching:   coaching_output = forward_coaching(x, role_id) # [B, output_dim]
7. Decode:     top_concepts = top-k(probs, k=3)               # interpretaveis

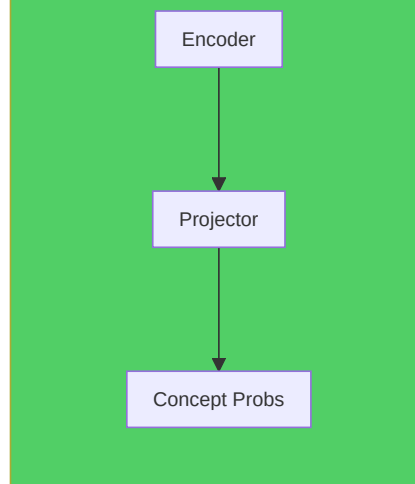
```

Output `forward_vl()` : Dicionario com 5 chaves:

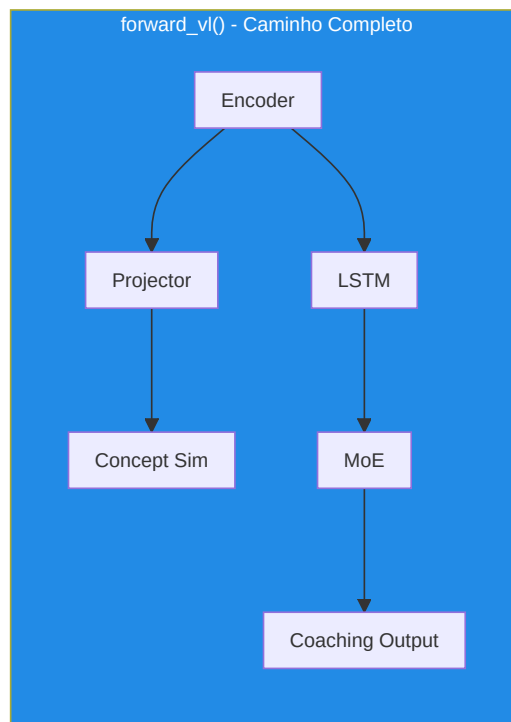
Chave	Forma	Conteudo
<code>concept_probs</code>	<code>[B, 16]</code>	Probabilidades softmax para cada conceito
<code>concept_logits</code>	<code>[B, 16]</code>	Scores de similaridade brutos (pre-softmax)
<code>top_concepts</code>	<code>List[tuple]</code>	<code>[(nome_conceito, probabilidade), ...]</code> para a primeira amostra
<code>coaching_output</code>	<code>[B, output_dim]</code>	Predicao coaching padrao (via pai)
<code>latent</code>	<code>[B, 256]</code>	Embedding latente pooled do encoder

Caminho leve - `get_concept_activations()` : Forward apenas conceitual sem cabeca de coaching nem LSTM. Utiliza `torch.no_grad()` para eficiencia maxima durante a inferencia.

get_concept_activations() - Apenas Conceitos



forward_vl() - Caminho Completo



ConceptLabeler: Dois Modos de Rotulagem

A classe `ConceptLabeler` gera **rotulos soft multi-label** ($[0, 1]^{16}$) para o treinamento VL-JEPA. Suporta dois modos:

Modo 1 - Baseado em Resultado (preferido, correcao G-01): `label_from_round_stats(round_stats)` gera rotulos a partir de dados de **resultado do round** (abates, mortes, danos, sobrevivencia, trade kill, utilidade, equipamento, round vencido). Esses dados sao **ortogonais** ao vetor de input de 25 dim, eliminando o label leakage.

Conceito	Sinal de Resultado Usado
<code>positioning_aggressive</code> (0)	<code>opening_kill=True</code> -> 0.8, <code>kills>=2+survived</code> -> 0.6
<code>positioning_passive</code> (1)	<code>survived</code> , <code>no opening</code> , <code>damage<60</code> -> 0.7
<code>positioning_exposed</code> (2)	<code>opening_death=True</code> -> 0.8, <code>deaths>0+damage<40</code> -> 0.6
<code>utility_effective</code> (3)	<code>utility_total>80</code> + <code>round_won</code> -> 0.5+util/300
<code>utility_wasteful</code> (4)	<code>zero utility</code> -> 0.5, <code>utility+lost</code> -> 0.4
<code>economy_efficient</code> (5)	<code>eco win (equip<2000)</code> -> 0.9, <code>normal win</code> -> 0.7
<code>economy_wasteful</code> (6)	<code>high equip+loss</code> -> 0.4+equip/16000
<code>engagement_favorable</code> (7)	<code>multi-kill+survived</code> -> 0.5+kills x 0.15
<code>engagement_unfavorable</code> (8)	<code>deaths+no kills+low dmg</code> -> 0.7
<code>trade_responsive</code> (9)	<code>trade_kills>0</code> -> 0.6+tk x 0.2
<code>trade_isolated</code> (10)	<code>died</code> , <code>not traded</code> , <code>no trade kills</code> -> 0.7
<code>rotation_fast</code> (11)	<code>assists>=1+round_won</code> -> 0.6+assists x 0.1
<code>information_gathered</code> (12)	<code>flashes>=2+survived</code> -> 0.6
<code>momentum_leveraged</code> (13)	<code>rating>1.5</code> -> <code>rating/2.5</code> , <code>kills>=3</code> -> 0.7
<code>clutch_composed</code> (14)	<code>kills>=2+survived+won</code> -> 0.6
<code>aggression_calibrated</code> (15)	<code>efficiency = kills x 1000/equip</code> -> <code>min(eff x 0.5, 1.0)</code>

Modo 2 - Heuristica Legacy (fallback com label leakage): `label_tick(features)` gera rotulos diretamente do vetor de features de 25 dim. Isso cria **label leakage** porque o modelo pode "trapacear" reconstruindo as features de input em vez de aprender patterns latentes. Usado apenas quando `RoundStats` nao esta disponivel, com um aviso de log uma unica vez.

`label_batch(features_batch)` : Wrapper que gerencia batches 2D `[B, 25]` e 3D `[B, seq_len, 25]` (media dos rotulos sobre a sequencia para input 3D).

Referencia indices feature (METADATA_DIM=25):

```

0: health/100      1: armor/100      2: has_helmet      3: has_defuser
4: equip/10000     5: is_crouching   6: is_scoped       7: is_blinded
8: enemies_vis     9: pos_x/4096     10: pos_y/4096     11: pos_z/1024
12: view_x_sin     13: view_x_cos    14: view_y/90      15: z_penalty
16: kast_est       17: map_id        18: round_phase
19: weapon_class   20: time_in_round/115  21: bomb_planted
22: teammates_alive/4  23: enemies_alive/5  24: team_economy/16000

```

Funcoes de Perda VL-JEPA

1. `jepa_contrastive_loss()` - InfoNCE (ja documentada acima)

Formula: $-\log(\exp(\text{sim}(\text{pred}, \text{target})/\tau) / (\exp(\text{sim}(\text{pred}, \text{target})/\tau) + \text{Sigma} \exp(\text{sim}(\text{pred}, \text{neg}_i)/\tau)))$ com $\tau=0.07$. A implementacao PyTorch utiliza um truque eficiente: concatena a similaridade positiva e as similaridades negativas em um vetor de logits `[pos_sim, neg_sim_1, ..., neg_sim_N]` e aplica `F.cross_entropy(logits, labels=0)` - onde o rotulo `0` indica que o primeiro elemento (a similaridade positiva) e a classe correta. Isso e matematicamente equivalente a formula InfoNCE mas explora a numericamente estavel `log_softmax` interna de PyTorch.

2. `vl_jepa_concept_loss()` - Alinhamento Conceitos + Diversidade VICReg

```

concept_loss = BCE_with_logits(concept_logits, concept_labels) # Multi-label
diversity_loss = -std(L2_normalize(concept_embeddings), dim=0).mean() # VICReg
total = alpha * concept_loss + beta * diversity_loss

```

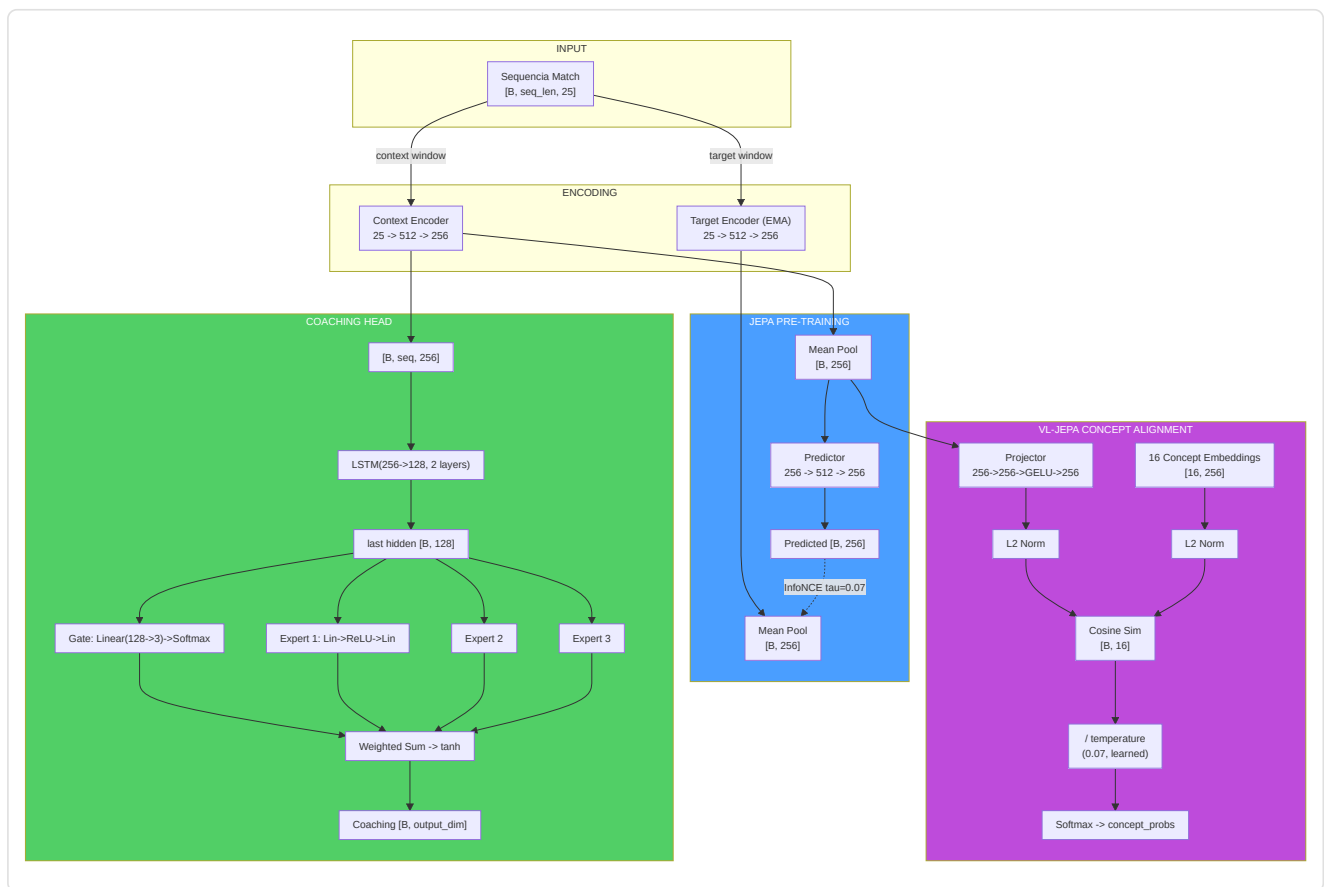
Termo	Formula	Peso Default	Proposito
<code>concept_loss</code>	<code>F.binary_cross_entropy_with_logits(logits, labels)</code>	$\alpha = 0.5$	Alinha embeddings aos conceitos corretos
<code>diversity_loss</code>	<code>-std_per_dim(L2_norm(concept_embs)).mean()</code>	$\beta = 0.1$	Impede o colapso dos embeddings de conceito

Perda total no training step VL-JEPA (`train_step_vl`):

```
L_total = L_inforce + alpha x L_concept + beta x L_diversity
```

Onde `L_inforce` e a perda contrastiva padrao JEPA e `(alpha x L_concept + beta x L_diversity)` e o termo de alinhamento conceitual.

Fluxo Dimensional Completo JEPA / VL-JEPA



JEPATrainer: Treinamento com Monitoramento Drift

Definido em `jepa_trainer.py`. Gerencia tanto o treinamento JEPA padrao quanto VL-JEPA, com retreinamento automatico baseado no drift.

Parametro	Default	Proposito
Optimizer	AdamW (lr=1e-4, weight_decay=1e-4)	Otimizacao com decaimento dos pesos
Scheduler	CosineAnnealingLR (T_max=100)	Decaimento ciclico do learning rate
DriftMonitor	z_threshold=2.5	Detecta drift das features alem de 2.5 sigma
drift_history	<code>List[DriftReport]</code>	Historico dos reports de drift

Codificacao de negativos compartilhada - `encode_raw_negatives(negatives, seq_len)` (NN-H-02):

Metodo compartilhado que codifica negativos brutos (feature space) no espaco latente. Quando o orchestrator envia negativos do cross-match pool (dimensao `METADATA_DIM`), esses devem ser transformados em embeddings latentes para o calculo da loss contrastiva. O metodo: reshape `[B*N, 1, D]` -> expand por `seq_len` -> encode via `target_encoder` com `torch.no_grad()` -> mean pool -> reshape `[B, N, latent_dim]`. Essa logica estava anteriormente duplicada entre trainer e orchestrator; a centralizacao (NN-H-02) previne desalinhamentos.

Ciclo de treinamento - `train_step(x_context, x_target, negatives)`:

1. Forward pass JEPA: `pred, target = model.forward_jepa_pretrain(context, target)`
2. **Auto-detect negativos brutos:** Se `negatives.shape[-1] != latent_dim`, os negativos sao features brutas -> sao auto-codificados via `encode_raw_negatives()` (NN-H-02)
3. Loss InfoNCE em embeddings normalizados
4. Backward + optimizer step
5. **Atualizacao EMA target encoder** (deve ocorrer DEPOIS de `optimizer.step()`)
6. **Monitoramento diversidade embedding (P9-02):** Calcula a variancia media das dimensoes latentes. Se `variance < 0.01`, emite um warning de potencial colapso dos embeddings - o modelo esta convergindo para uma representacao degenerada onde todos os inputs produzem o mesmo vetor

Guarda batch degenerado (NN-JT-01): Em modo in-batch negatives, se `batch_size < 2` o batch e pulado - nao e possivel construir negativos excluindo a si mesmo de um batch de um unico elemento. Isso previne erros na indexacao dos negativos durante as fases iniciais do treinamento com poucos dados.

Training step VL-JEPA - `train_step_vl()`: Estende `train_step` com:

1. Forward pass JEPA padrao (InfoNCE)
2. Forward VL: `model.forward_vl(x_context)` -> concept_logits
3. **Geracao de rotulos (preferencia G-01):** Se `round_stats` estiver disponivel -> `label_from_round_stats()` (no leakage). Caso contrario -> `label_batch()` (heuristica legacy com aviso uma unica vez)
4. Concept loss + diversity loss: `vl_jepa_concept_loss(logits, labels, embeddings, alpha, beta)`
5. Loss total: `L_infonce + L_concept_total`
6. Backward + optimize + EMA update

Output: `{total_loss, infonce_loss, concept_loss, diversity_loss}`.

Monitoramento drift - `check_val_drift(val_df, reference_stats)`:

- Utiliza `DriftMonitor` da pipeline de validacao
- Calcula z-score para cada feature do validation set em relacao as estatisticas de referencia
- Se `max_z_score > 2.5`, o report marca `is_drifted=True`
- `should_retrain(drift_history, window=5)` -> se a maioria das ultimas 5 janelas mostra drift, ativa o flag `_needs_full_retrain`

Retreinamento automatico - `retrain_if_needed(full_data_loader, device, epochs=10)`:

- Se o flag `_needs_full_retrain` estiver ativo, reseta o scheduler e reexecuta `epochs` epocas completas
- Depois do treinamento, cancela o flag e o historico drift
- Retorna `True/False` para indicar se o treinamento ocorreu

Pipeline de Treinamento Standalone (`jepa_train.py`)

Script standalone para pre-treinamento e fine-tuning JEPA, executavel via CLI:

```
python -m Programma_CS2_RENAN.backend.nn.jepa_train --mode pretrain
python -m Programma_CS2_RENAN.backend.nn.jepa_train --mode finetune --model-path models/jepa_model.pt
```

JEPAPretrainDataset : Dataset PyTorch para o pre-treinamento:

Parametro	Default	Descricao
<code>context_len</code>	10	Comprimento janela contexto (tick)
<code>target_len</code>	10	Comprimento janela target (tick)
<code>match_sequences</code>	<code>List[np.ndarray]</code>	Sequencias de match <code>[num_rounds, METADATA_DIM]</code>

Para cada amostra, seleciona um ponto de partida aleatorio na sequencia e retorna `{"context": [context_len, 25], "target": [target_len, 25]}`.

Nota (F3-25): O ponto de partida usa `np.random.randint()` com estado global nao seedado -> janelas nao reproduzíveis entre runs. Para treinamento determinístico, usar `worker_init_fn` ou um `Generator` dedicado no `DataLoader`.

`_roundstats_to_features(rs: RoundStats) -> List[float]` : Extrai um vetor de **16 features** de uma unica linha `RoundStats`: `[kills, deaths, damage_dealt/100, headshot_kills, assists, trade_kills, was_traded, opening_kill, opening_death, he_damage/100, molotov_damage/100, flashes_thrown, smokes_thrown, equipment_value/5000, round_rating, side_CT]`. O vetor e entao padded para `METADATA_DIM` (25) com zeros.

Exclusao `round_won` (P-RSB-03): O campo `round_won` e **deliberadamente excluido** do vetor feature. Inclui-lo causaria data leakage: o modelo veria o resultado do round nos dados de input, permitindo-lhe prever banalmente os resultados a partir do proprio resultado. `round_won` e corretamente utilizado como **label** em `label_from_round_stats()` (`jepa_model.py`), onde gera os rotulos de supervisao para o ramo VL-JEPA.

`_MIN_ROUNDS_FOR_SEQUENCE = 6` : Requisito minimo de rounds para construir uma sequencia de treinamento valida. Partidas com menos de 6 rounds nao fornecem contexto temporal suficiente para aprender patterns taticos significativos e sao descartadas silenciosamente.

`load_pro_demo_sequences(limit=100)` : Carrega sequencias demo profissionais do banco de dados. Utiliza `_roundstats_to_features()` para extrair features reais por-round de `RoundStats`, com fallback para 12 features agregadas em nivel de match de `PlayerMatchStats` (padded para `METADATA_DIM` com zeros) apenas quando `RoundStats` nao estiver disponivel.

Aviso Critico (F3-08): No caminho de fallback match-aggregate, o script usa `np.tile(features, (20, 1))` para criar 20 frames identicos a partir de um unico vetor agregado. Isso torna o pre-treinamento JEPA **uma operacao identidade** - o modelo aprende simplesmente a copiar o input, nao as dinamicas temporais. O caminho primario com `RoundStats` reais e o `TrainingOrchestrator` no caminho de producao **nao sao afetados** por esse problema e usam dados por-round/por-tick reais.

`train_jepa_pretrain()` : 50 epocas, `batch_size=16`, `lr=1e-4`, 8 negativos in-batch. O optimizer inclui SOMENTE `context_encoder` e `predictor` - o `target_encoder` e atualizado exclusivamente via EMA.

`train_jepa_finetune()` : 30 epocas, `batch_size=16`, `lr=1e-3`, `weight_decay=1e-3`. Congela os encoders e otimiza apenas LSTM + MoE + Gate.

Persistencia: `save_jepa_model()` salva `{model_state_dict, is_pretrained}`. `load_jepa_model()` carrega com `weights_only=True` (seguranca).

SuperpositionLayer - Gating Contextual (`layers/superposition.py`)

Modulo standalone que implementa uma camada linear com **gating dependente do contexto**, usado dentro do RAP Coach Strategy Layer.

```
class SuperpositionLayer(nn.Module):
    def __init__(self, in_features, out_features, context_dim=METADATA_DIM):
        self.weight = nn.Parameter(empty(out_features, in_features))
        nn.init.kaiming_uniform_(self.weight, a=math.sqrt(5)) # P1-09: Kaiming init
        self.bias = nn.Parameter(zeros(out_features))
        self.context_gate = nn.Linear(context_dim, out_features) # Superposition Controller

    def forward(self, x, context):
        gate = sigmoid(self.context_gate(context)) # [B, out_features]
        self._last_gate_live = gate # Com gradiente (para sparsity loss)
        self._last_gate_activations = gate.detach() # Copia detached (para observabilidade)
        out = F.linear(x, self.weight, self.bias)
        return out * gate # Modulacao contextual
```

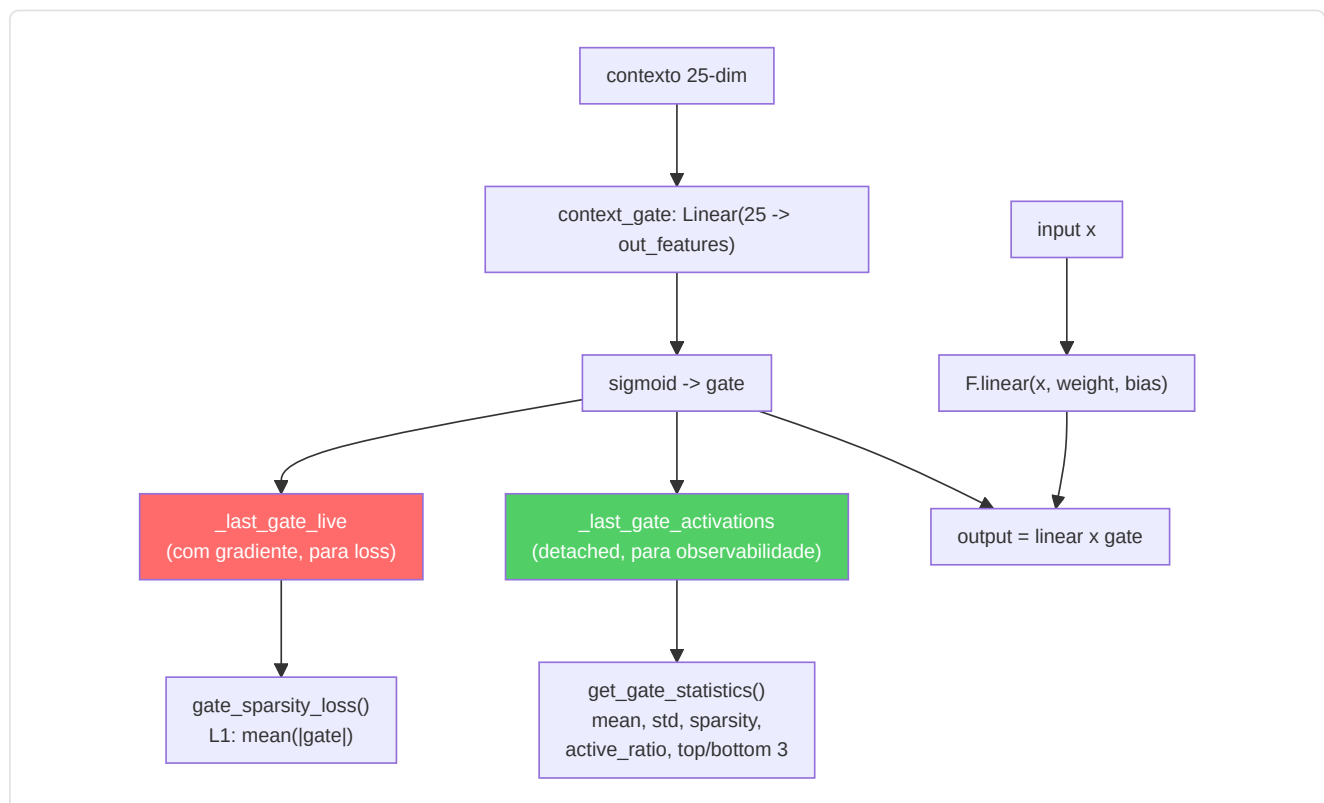
Mecanismo: O output de cada neurônio é multiplicado por um gate sigmoide condicionado nas features de contexto (25-dim). Isso permite ao modelo "acender" ou "apagar" neurônios dinamicamente com base na situação de jogo.

Inicializacao Kaiming (P1-09): Os pesos são inicializados com `kaiming_uniform_` (distribuição Kaiming He, 2015) em vez de `torch.randn()`. Essa inicialização garante que a variância dos pesos seja proporcional ao fan-in da camada, prevenindo o desaparecimento ou a explosão dos gradientes nas redes profundas. O parâmetro `a=math.sqrt(5)` é o valor padrão para camadas lineares em PyTorch.

Design dual-tensor (NN-24): O gate sigmoide é memorizado em **duas cópias separadas** durante cada forward pass:

Tensor	Gradiente	Proposito
<code>_last_gate_live</code>	Sim (mantém o grafo computacional)	Usado por <code>gate_sparsity_loss()</code> para backpropagation - o gradiente flui através do gate para <code>context_gate</code>
<code>_last_gate_activations</code>	Não (detached)	Usado por <code>get_gate_statistics()</code> para observabilidade - nenhum custo de memória para o grafo

Essa separação resolve o conflito entre a necessidade de gradientes (para a loss de sparsidade) e a necessidade de observabilidade leve (para logging e TensorBoard).



Observabilidade integrada:

Metodo	Retorno	Descricao
<code>get_gate_activations()</code>	<code>Tensor</code> ou <code>None</code>	Ultimas ativacoes do gate (<code>_last_gate_activations</code> , detached)
<code>get_gate_statistics()</code>	<code>Dict[str, float]</code>	Estatisticas completas do gate (veja tabela abaixo)
<code>gate_sparsity_loss()</code>	<code>Tensor</code>	Perda L1 <code>mean()</code>
<code>enable_tracing(interval)</code>	-	Log detalhado do gate a cada <code>interval</code> passos
<code>disable_tracing()</code>	-	Restaura intervalo de logging para 100

Campos de `get_gate_statistics()` :

Campo	Tipo	Significado
<code>mean_activation</code>	float	Media das ativacoes do gate no batch
<code>std_activation</code>	float	Desvio padrao das ativacoes
<code>sparsity</code>	float	Fracao de dimensoes com media < 0.1 (mais alto = mais esperso)
<code>active_ratio</code>	float	Fracao de dimensoes com media > 0.5 (mais alto = mais ativo)
<code>top_3_dims</code>	List[int]	As 3 dimensoes do gate mais ativas
<code>bottom_3_dims</code>	List[int]	As 3 dimensoes do gate menos ativas

Log periodico durante treinamento: A cada 100 forward pass (configuravel via `enable_tracing(interval)`), logga via logger estruturado: dimensoes ativas (gate_mean > 0.5), dimensoes esparsas (gate_mean < 0.1) e media geral.

Modulo EMA Standalone

A atualizacao **Exponential Moving Average** do target encoder e implementada diretamente em

`JEPACoachingModel.update_target_encoder(momentum=0.996)` :

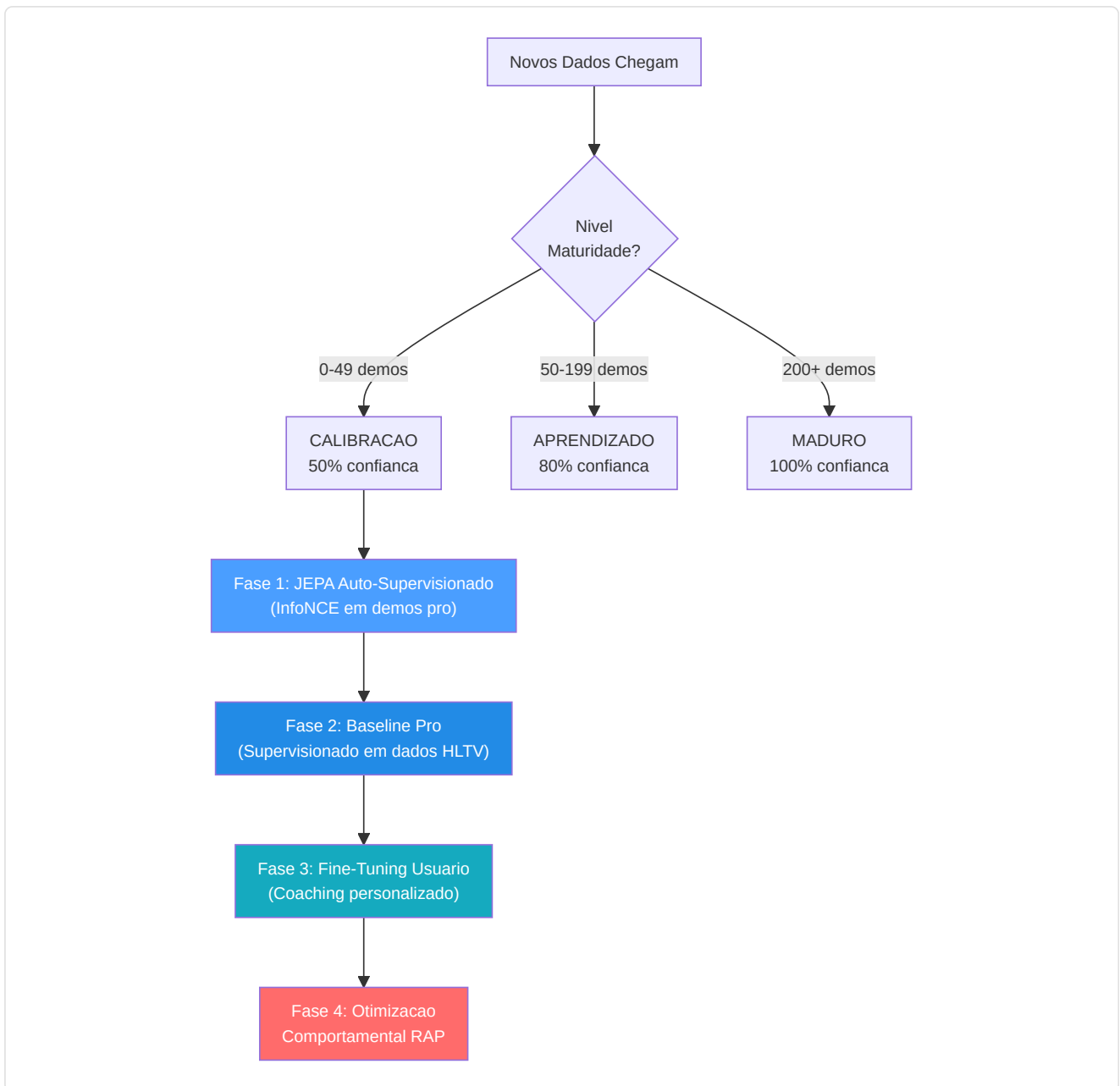
```
with torch.no_grad():
    for param_q, param_k in zip(context_encoder.parameters(), target_encoder.parameters()):
        param_k.data = param_k.data * momentum + param_q.data * (1.0 - momentum)
```

Invariantes:

- A atualizacao EMA ocorre **sempre depois** de `optimizer.step()` - nunca antes, caso contrario os gradientes ainda nao estao aplicados
- O target encoder **nunca recebe gradientes** diretos - apenas atualizacoes EMA
- O momentum 0.996 significa que o target encoder "absorve" apenas 0,4% dos pesos do encoder online a cada passo - atualizacao muito conservadora
- `state_dict()` do modelo retorna tensores **clonados** (`.clone()`) para prevenir aliasing acidental - um bug real corrigido durante o audit onde `state_dict()` retornava referencias diretas aos tensores do modelo em vez de copias, causando corrupcao quando o chamador modificava o dicionario

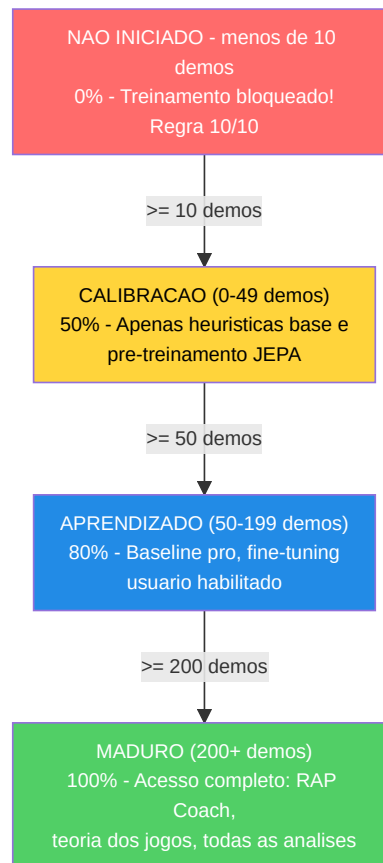
-CoachTrainingManager (Orquestracao)

Definido em `coach_manager.py` . Este e o **cerebro do processo de formacao**, que gerencia um rigoroso **ciclo de formacao de 3 niveis, baseado na maturidade**, dividido em 4 fases:



Níveis de maturidade e multiplicadores de confiança:

Nível	Contagem demos	Multiplicador de confiança	Funcionalidades desbloqueadas
CALIBRACAO	0-49	0,50	Heurística de base, pre-treinamento JEPA
APRENDIZADO	50-199	0,80	Comparacao base profissional, otimizacao usuario
MADURO	200+	1,00	Coach RAP completo, teoria dos jogos, analise completa



Pre-requisitos (Regra 10/10): Requer ≥ 10 demos profissionais OU (≥ 10 demos usuário + conta Steam/FACEIT conectada) antes de iniciar qualquer treinamento.

O manager utiliza um **contrato de treinamento** rigoroso com 25 funcionalidades (correspondentes a `METADATA_DIM`).

Problema resolvido (ex G-10): `coach_manager.py` agora define `TRAINING_FEATURES` com os nomes canonicos corretos para todos os 25 indices, alinhados perfeitamente com `vectorizer.py`. A assercao `len(TRAINING_FEATURES) == METADATA_DIM` e valida e todos os nomes estao atualizados. Alem disso, `MATCH_AGGREGATE_FEATURES` define as 25 features agregadas em nivel de partida: `["avg_kills", "avg_deaths", "avg_adr", "avg_hs", "avg_kast", "kill_std", "adr_std", "kd_ratio", "impact_rounds", "accuracy", "econ_rating", "rating", "opening_duel_win_pct", "clutch_win_pct", "trade_kill_ratio", "flash_assists", "positional_aggression_score", "kpr", "dpr", "rating_impact", "rating_survival", "he_damage_per_round", "smokes_per_round", "unused_utility_per_round", "thrusmoke_kill_pct"]`. Ambas as listas sao validadas em runtime: se uma das duas tiver um comprimento diferente de `METADATA_DIM`, o modulo levanta `ValueError` no momento da importacao.

```

health, armor, has_helmet, has_defuser, equipment_value,
is_crouching, is_scoped, is_blinded,
enemies_visible,
pos_x, pos_y, pos_z,
view_yaw_sin, view_yaw_cos, view_pitch,
z_penalty, kast_estimate, map_id, round_phase,
weapon_class, time_in_round, bomb_planted,
teammates_alive, enemies_alive, team_economy

```

Indices target: `TARGET_INDICES = list(range(OUTPUT_DIM)) = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]` - o modelo preve deltas de melhoria para as primeiras **10 metricas agregadas** em nivel de partida: `[avg_kills, avg_deaths, avg_adr, avg_hs, avg_kast, kill_std, adr_std, kd_ratio, impact_rounds, accuracy]`.

-TrainingOrchestrator

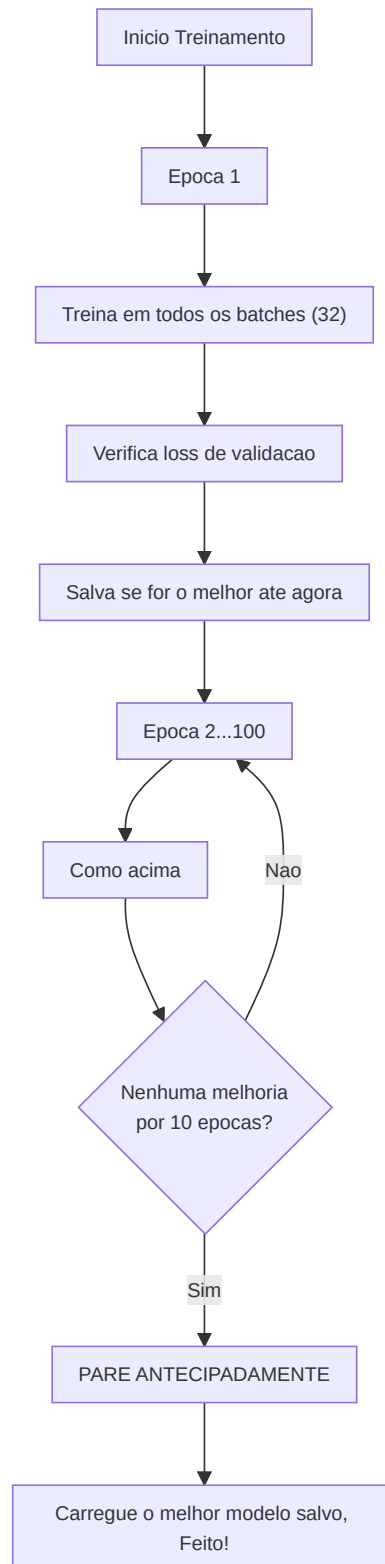
Definido em `training_orchestrator.py`. Ciclo de epocas unificado, validacao, parada antecipada e checkpoint para os modelos JEPA, VL-JEPA, RAP e RAP Lite.

Parametro	Predefinido	Proposito
<code>model_type</code>	"jepa"	Caminhos para o trainer JEPA, VL-JEPA, RAP ou RAP Lite
<code>max_epochs</code>	100	Limite maximo de treinamento
<code>patience</code>	10	Paciencia na parada antecipada
<code>batch_size</code>	32	Amostras por batch
<code>callbacks</code>	None	Lista de instancias de <code>TrainingCallback</code> para a integracao com Observatory

O orchestrator se integra com Observatory atraves de `CallbackRegistry`. Dispara eventos do ciclo de vida em **5 pontos**: `on_train_start` (antes da primeira epoca), `on_epoch_start` (inicio de cada epoca), `on_batch_end` (depois de cada batch de treinamento, inclui outputs de perda e trainer), `on_epoch_end` (depois da validacao, inclui modelo e perdas), `on_train_end` (depois da conclusao do treinamento ou interrupcao antecipada). Quando nenhuma callback e registrada, todas as chamadas `fire()` sao operacoes sem custo. Os erros de callback sao detectados e registrados, sem nunca causar o travamento do ciclo de treinamento.

Pool negativos cross-match (NN-H-03): O orchestrator mantem um pool de feature vectors de batches anteriores (`_neg_pool`, max 500 vetores). Os negativos contrastivos sao amostrados deste pool em vez do batch atual, garantindo que os negativos venham de **partidas diferentes** e nao da mesma sequencia temporal de contexto/target. Quando o pool ainda esta vazio (warm-up), o sistema recai em in-batch sampling. Isso evita falsos negativos: dois ticks da mesma acao de jogo seriam muito similares para serem negativos uteis.

Gate de qualidade pre-treinamento (P3-D): Antes de iniciar qualquer treinamento, o orchestrator executa `run_pre_training_quality_check()`. Se o report de qualidade nao passar (dados insuficientes, distribuicoes anormais, features ausentes), o treinamento e **abortado** com um log de erro explicativo. Isso impede de desperdicar GPU em dados que produziram um modelo inutil.



-ModelFactory e Persistencia

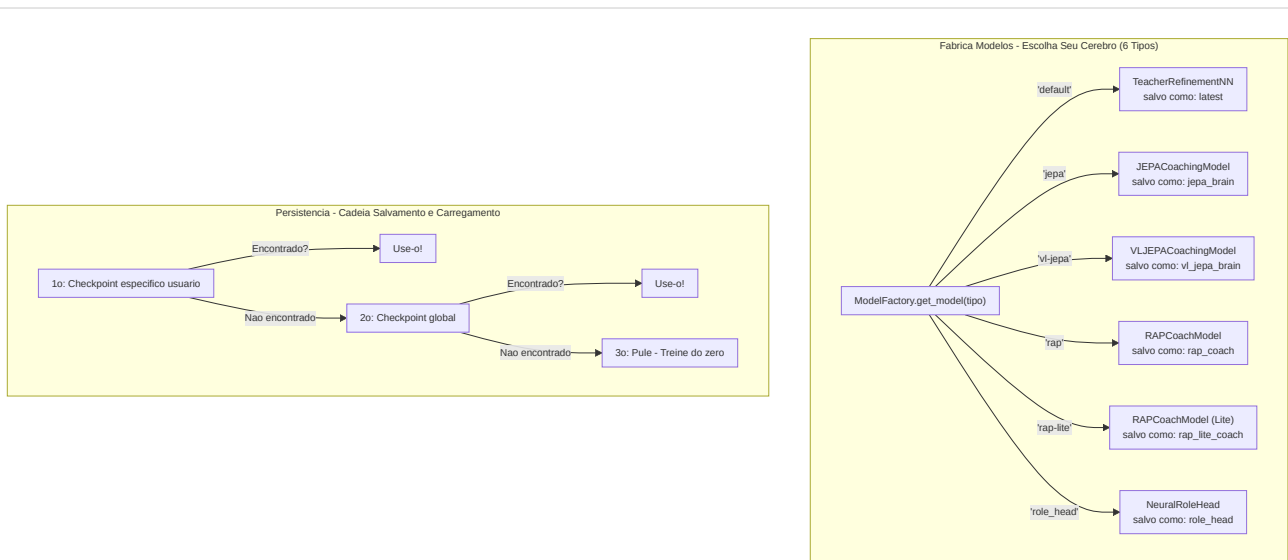
ModelFactory (`factory.py`) fornece uma instanciação unificada do modelo:

Tipo Constante	Classe Modelo	Nome Checkpoint	Configuracoes predefinidas de fabrica
<code>TYPE_LEGACY</code> ("default")	<code>TeacherRefinementNN</code>	"latest"	<code>input_dim=METADATA_DIM(25)</code> , <code>output_dim=OUTPUT_DIM(10)</code> , <code>hidden_dim=HIDDEN_DIM(128)</code>
<code>TYPE_JEPA</code> ("jepa")	<code>JEPACoachingModel</code>	"jepa_brain"	<code>input_dim=METADATA_DIM(25)</code> , <code>output_dim=OUTPUT_DIM(10)</code>
<code>TYPE_VL_JEPA</code> ("vl-jepa")	<code>VLJEPACoachingModel</code>	"vl_jepa_brain"	<code>input_dim=METADATA_DIM(25)</code> , <code>output_dim=OUTPUT_DIM(10)</code>
<code>TYPE_RAP</code> ("rap")	<code>RAPCoachModel</code>	"rap_coach"	<code>metadata_dim=METADATA_DIM(25)</code> , <code>output_dim=10</code>
<code>TYPE_RAP_LITE</code> ("rap-lite")	<code>RAPCoachModel</code>	"rap_lite_coach"	<code>metadata_dim=METADATA_DIM(25)</code> , <code>output_dim=10</code> , <code>use_lite_memory=True</code>
<code>TYPE_ROLE_HEAD</code> ("role_head")	<code>NeuralRoleHead</code>	"role_head"	<code>input_dim=5</code> , <code>hidden_dim=32</code> , <code>output_dim=5</code>

Nota (P1-08): Em uma versao anterior, a factory utilizava `output_dim=4` e `hidden_dim=64` para os modelos legacy, criando um desalinhamento com `CoachNNConfig`. Isso foi corrigido: agora todos os modelos de coaching (Legacy, JEPA, VL-JEPA) utilizam `OUTPUT_DIM = 10` - as primeiras 10 features core agregadas sobre as quais o modelo preve ajustes delta. `HIDDEN_DIM = 128` esta alinhado tanto em `config.py` quanto em `factory.py`. O modelo RAP e RAP Lite compartilham a mesma classe `RAPCoachModel`, mas RAP Lite ativa `use_lite_memory=True`, substituindo a memoria LTC-Hopfield com um fallback LSTM puro em PyTorch (util quando as dependencias `ncps` / `hflayers` nao estao disponiveis). O modelo RAP e importado do caminho canonico `backend/nn/experimental/rap_coach/model.py` (o antigo `backend/nn/rap_coach/model.py` e um shim de redirecionamento).

StaleCheckpointError: Se as dimensoes de um checkpoint salvo nao corresponderem a configuracao atual do modelo (por exemplo, depois de uma atualizacao de `output_dim=4` para `output_dim=10`), o sistema levanta `StaleCheckpointError` em vez de carregar silenciosamente pesos incompativeis, prevenindo corrupcoes silenciosas.

Persistencia (`persistence.py`): Salva/carrega modelos com `weights_only=True` (seguranca -- previne execucao arbitraria de codigo durante a deserializacao de checkpoint). Implementa um **registro hash SHA-256** para validacao de integridade dos checkpoint: cada arquivo salvo e acompanhado por um arquivo sidecar `.sha256` contendo o hash do checkpoint; no carregamento, o sistema verifica que o hash corresponda antes de prosseguir com a deserializacao. **Cadeia de fallback de 3 niveis:** (1) checkpoint especifico do usuario, (2) checkpoint global, (3) pule -- treine do zero. Gerencia dimensoes nao correspondentes via `StaleCheckpointError`.



-Configuracao (config.py)

```
GLOBAL_SEED = 42                                # Reprodutibilidade global (AR-6, P1-02)
INPUT_DIM = METADATA_DIM = 25                    # Vetor canonico de 25 dimensoes (era 19, era legacy 12)
OUTPUT_DIM = 10                                  # As primeiras 10 features core sobre as quais o modelo preve ajustes
HIDDEN_DIM = 128                                  # Dimensao oculta para AdvancedCoachNN / TeacherRefinementNN
BATCH_SIZE = 32
LEARNING_RATE = 0.001
EPOCHS = 50
RAP_POSITION_SCALE = 500.0                        # P9-01: Fator de escala para delta posicao ([-1,1] -> unidades mundo)
```

Nota: `INPUT_DIM` e importado de `feature_engineering/__init__.py` onde `METADATA_DIM = 25`. `OUTPUT_DIM = 10` define o numero de features sobre as quais o modelo produz predicoes de ajuste - as primeiras 10 features agregadas de match (avg_kills, avg_deaths, avg_adr, avg_hs, avg_kast, kill_std, adr_std, kd_ratio, impact_rounds, accuracy). Essa escolha de design concentra a capacidade preditiva do modelo nas metricas de desempenho mais acionaveis, em vez de tentar prever todas as 25 features (muitas das quais sao contextuais e nao diretamente melhoraveis pelo jogador). `RAP_POSITION_SCALE = 500.0` e o fator canonico para converter os outputs normalizados do modelo RAP (no intervalo [-1, 1]) em deslocamentos nas unidades mundo CS2.

Nota arquitetural: A `CoachNNConfig` dataclass em `model.py` define `output_dim = METADATA_DIM` (25) como default, mas a `ModelFactory` sempre sobrescreve esse valor com `OUTPUT_DIM = 10` durante a instanciacao. O `output_dim` efetivo em producao para todos os modelos (Legacy, JEPA, VL-JEPA, RAP, RAP Lite) e portanto **10**, nao 25. Historicamente, `OUTPUT_DIM` era 4 (4 metricas selecionadas), depois foi elevado para 10 para cobrir as features agregadas mais relevantes.

Gerenciamento de dispositivos: `get_device()` implementa uma **selecao GPU inteligente de 3 niveis**:

1. **Override usuario:** Se configurado `CUDA_DEVICE` (ex. "cuda:0" ou "cpu"), usa esse
2. **GPU discreta automatica:** `_select_best_cuda_device()` enumera todos os dispositivos CUDA e seleciona aquele com mais VRAM, **penalizando as GPUs integradas** (Intel UHD, Iris) atraves de keyword matching. Em sistemas multi-GPU (ex. Intel UHD + NVIDIA GTX 1650), a GPU discreta vence sempre
3. **Fallback CPU:** Se nenhuma GPU CUDA estiver disponivel

Dimensionamento batch baseado na intensidade ML: `Alto=128`, `Medio=32`, `Baixo=8`. O atraso de throttling entre batches se adapta: `Alto=0.0s`, `Medio=0.05s`, `Baixo=0.2s`.

-Controle Qualidade Dados Pre-Treinamento (data_quality.py)

Antes de cada ciclo de treinamento, o sistema executa uma verificacao de qualidade dos dados disponiveis atraves de `run_pre_training_quality_check()`. Este gate previne treinamento com dados insuficientes ou corrompidos.

DataQualityReport dataclass:

Campo	Tipo	Descricao
<code>total_tick_rows</code>	<code>int</code>	Total de linhas em <code>PlayerTickState</code>
<code>train_rows / val_rows / test_rows</code>	<code>int</code>	Linhas por split (TRAIN/VAL/TEST)
<code>zero_position_rate</code>	<code>float</code>	Fracao de ticks com posicao (0,0,0) -- indicador de dados corrompidos
<code>nan_rate</code>	<code>float</code>	Fracao de ticks com valores NaN
<code>round_outcome_distribution</code>	<code>Dict[str,int]</code>	Distribuicao de classes de resultado para verificacao de balanceamento
<code>complete_matches / incomplete_matches</code>	<code>int</code>	Contagem de partidas completas vs incompletas
<code>issues</code>	<code>List[str]</code>	Lista de problemas detectados
<code>passed</code>	<code>bool</code>	Veredito final (True = treinamento permitido)

5 verificacoes realizadas:

1. Contagem total de linhas de `PlayerTickState`
2. Distribuicao por `DatasetSplit` (TRAIN/VAL/TEST) de `PlayerMatchStats`
3. Taxa de posicao zero -- amostra ate 10.000 ticks para eficiencia
4. Completude das partidas via `MatchDataManager`
5. **Veredito:** falha se `total_tick_rows < min_samples` (padrao: 1000), `zero_position_rate > threshold` (padrao: 0.10), ou `train_rows == 0`

-Controlador de Treinamento (`training_controller.py`)

Controla **quando** o Coach pode treinar, gerenciando deduplicacao de demos, diversidade de dados e cota mensal.

`TrainingDecision` **dataclass:**

Campo	Tipo	Descricao
<code>should_train</code>	<code>bool</code>	Se deve prosseguir com o treinamento
<code>reason</code>	<code>str</code>	Motivacao legivel para humanos
<code>diversity_score</code>	<code>float</code>	Score de diversidade (0.0--1.0)

Classe `TrainingController` :

- `MIN_DIVERSITY_SCORE = 0.3` -- limiar minimo de diversidade para prosseguir
- `MAX_DEMOS_PER_MONTH` -- cota mensal das configuracoes (padrao: 10, NN-M-13)

Pipeline de decisao `should_train_on_demo(demo_path, match_stats) -> TrainingDecision` :

1. **Verificacao cota mensal** -- `_get_monthly_training_count()` conta `PlayerMatchStats` com `processed_at >= 30 dias atras`
2. **Calculo diversidade** -- `_calculate_diversity_score(new_stats)` calcula `1 - mean_cosine_similarity` contra as ultimas 5 partidas. Primeira partida: score = 1.0
3. **Decisao** -- se abaixo da cota E diversidade `>= 0.3`, `should_train = True`

Calculo de diversidade: `_extract_features(stats)` extrai um vetor de 6 dimensoes com z-scaling aproximado: kills, deaths, ADR, HS%, utility blind time, opening duel win %. A similaridade cosseno entre o novo vetor e cada um dos ultimos 5 previne treinamento com dados muito semelhantes (overfitting em um tipo de partida).

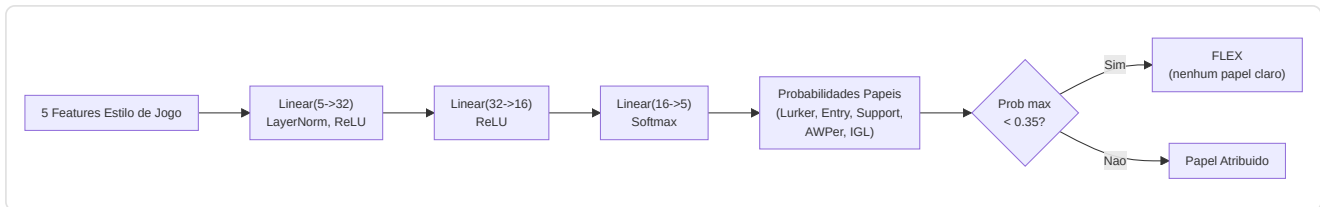
-NeuralRoleHead (MLP para a classificacao dos papeis)

Definido em `role_head.py`. Um MLP leve que preve as probabilidades de papel dos jogadores com base em 5 parametros de estilo de jogo, operando como **opinioao secundaria** junto com a heuristica `RoleClassifier`. A logica de consenso em `role_classifier.py` une ambas as opinioes para produzir a classificacao final.

Arquitetura:

```
Input (5 caracteristicas) -> Linear(5, 32) -> LayerNorm(32) -> ReLU
-> Linear(32, 16) -> ReLU
-> Linear(16, 5) -> Softmax -> 5 probabilidades de papel
```

~750 parametros aprendiveis. Custo de calculo minimo, adequado para a inferencia por partida.



Características de input (5 dimensoes):

#	Característica	Fonte	Intervalo	Significado
0	TAPD	<code>rounds_survived / rounds_played</code>	[0, 1]	Taxa de sobrevivencia - mais alta = mais passivo/de suporte
1	OAP	<code>entry_frgs / rounds_played</code>	[0, 1]	Agressividade inicial - mais alta = fragger em entrada
2	PODT	<code>was_traded_ratio</code>	[0, 1]	Percentual de mortes trocadas - mais alta = trocadas/iniciadas
3	rating_impact	<code>impact_rating</code> ou HLTV 2.0	float	Impacto geral nos rounds
4	aggression_score	<code>positional_aggression_score</code>	float	Tendencia para posicao avancada

Papeis de output (softmax de 5 dimensoes):

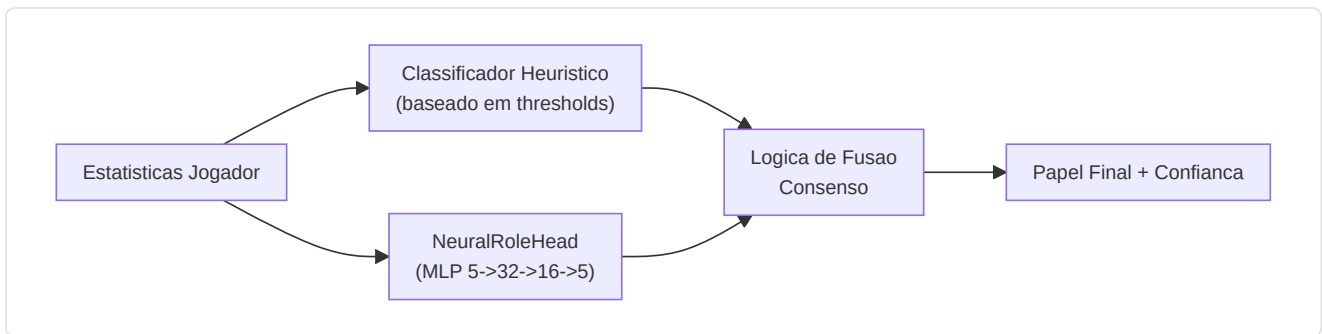
Indice	Papel	Descricao
0	LURKER	Se esconde atras das linhas inimigas
1	ENTRY_FRAGGER	Primeiro a entrar, enfrenta os duelos iniciais
2	SUPPORT	Ancoragem do site, uso das utilidades, trocas
3	AWPER	Especialista sniper
4	IGL	Lider em jogo, responsavel tatico

Threshold FLEX: Se `max(probabilidades) < 0,35`, o jogador e classificado como **FLEX** (versatil, nenhum papel dominante). Isso impede o modelo de forcar um papel quando o jogador e realmente um generalista.

Detalhes treinamento:

Aspecto	Valor
Perda	<code>KLDivLoss(reduction="batchmean")</code> nas previsoes log-softmax em relacao aos targets soft label
Suavizacao dos rotulos	<code>epsilon = 0,02</code> (impede <code>log(0)</code> , adiciona a regularizacao)
Otimizador	AdamW (<code>lr=1e-3</code> , <code>weight_decay=1e-4</code>)
Parada antecipada	Paciencia = 15 epocas sobre a perda de validacao
Epocas maximas	200
Divisao Train/Val	80/20 aleatorio (dados transversais, nao sequenciais)
Amostras minimas	20 (da tabela <code>Ext_PlayerPlaystyle</code>)
Fonte dados	<code>cs2_playstyle_roles_2024.csv</code> -> Tabela DB <code>Ext_PlayerPlaystyle</code>
Normalizacao	Media/std por funcionalidade calculada no momento do treinamento, salva em <code>role_head_norm.json</code>

Consenso com classificador heuristico (`role_classifier.py`):



- **Ambos concordam** -> confiança aumentada em +0,10
- **Em desacordo, margem neural > 0,1** -> vence a opinião neural
- ****Em desacordo, margem neural <= 0,1**** -> vence a opinião heurística
- **Neural não disponível** (nenhum checkpoint ou norm_stats) -> apenas heurística
- **Proteção cold-start** -> retorna FLEX com 0% de confiança se os thresholds não foram aprendidos

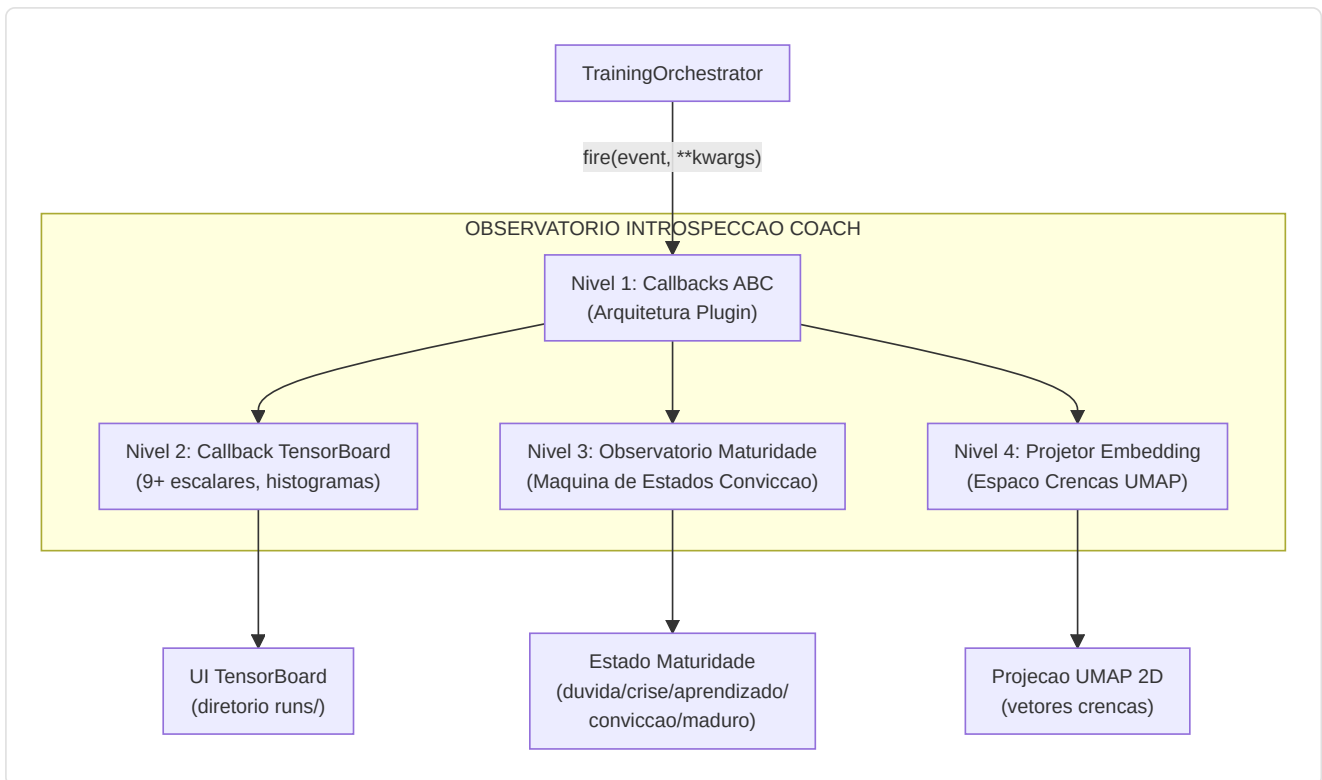
-Coach Introspection Observatory

Arquivos: `training_callbacks.py`, `tensorboard_callback.py`, `maturity_observatory.py`, `embedding_projector.py`

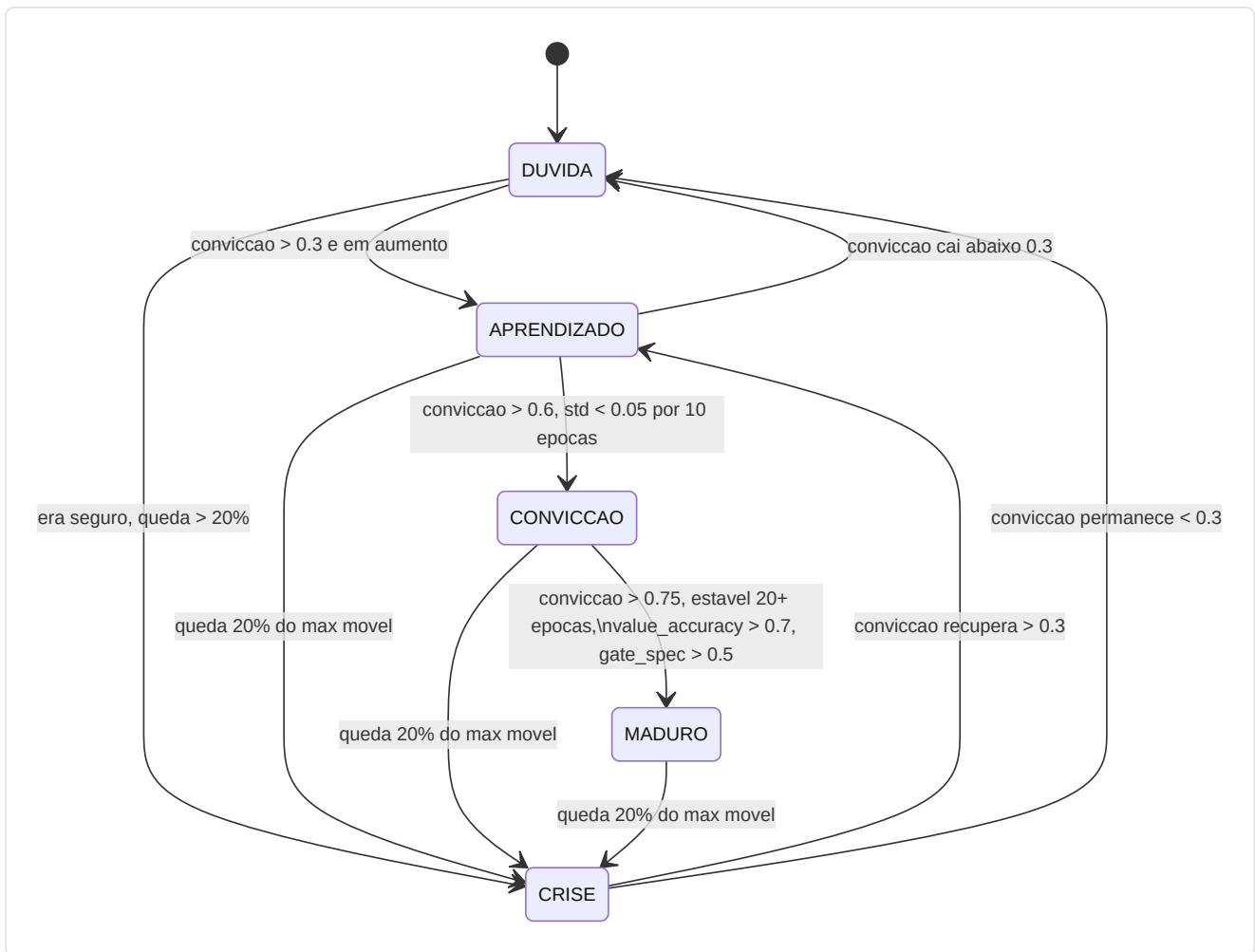
O Observatório é uma **arquitetura de plugin de 4 níveis** que instrumentaliza o ciclo de treinamento sem modificar o código de treinamento principal. Monitora os sinais neurais do treinador durante o treinamento e os traduz em estados de maturidade interpretáveis pelo humano, permitindo a desenvolvedores e operadores entender se o modelo está confuso, em fase de aprendizado ou pronto para a produção.

Arquitetura de 4 níveis:

Nível	Arquivo	Propósito	Output chave
1. Callback ABC	<code>training_callbacks.py</code>	Interface plugin + registro dispatch	<code>TrainingCallback ABC</code> , <code>CallbackRegistry.fire()</code>
2. TensorBoard	<code>tensorboard_callback.py</code>	Registro escalar + histograma	Mais de 9 sinais escalares, histogramas parâmetros/grad, histogramas gate/crenças/conceito
3. Maturidade	<code>maturity_observatory.py</code>	Máquina de estados de convicção	5 sinais -> <code>conviction_index</code> -> 5 estados de maturidade
4. Embedding	<code>embedding_projector.py</code>	Projeção crença/conceito UMAP	Figuras UMAP 2D interativas (degradação gradual se umap-learn não estiver instalado)

**Maturity State Machine:**

O **MaturityObservatory** calcula um **índice de conviccao** composto de 5 sinais neurais, o suaviza com EMA ($\alpha=0,3$) e classifica o modelo em um dos 5 estados de maturidade:



5 Sinais de Maturidade:

Sinal	Peso	Intervalo	O Que Mede	Fonte
<code>belief_entropy</code>	0,25	[0, 1]	Entropia de Shannon do vetor de crenca de 64 dim (mais baixo = mais seguro)	<code>model._last_belief_batch</code>
<code>gate_specialization</code>	0,25	[0, 1]	<code>1 - mean_gate_activation</code> (mais alto = especialistas mais especializados)	<code>SuperpositionLayer.get_gate_statistics()</code>
<code>concept_focus</code>	0,20	[0, 1]	<code>1 - entropy(concept_embedding_norms)</code> (entropia mais baixa = focalizado)	<code>model.concept_embeddings</code>
<code>value_accuracy</code>	0,20	[0, 1]	<code>1 - (val_loss / initial_val_loss)</code> (mais alto = melhor calibracao)	Ciclo de validacao
<code>role_stability</code>	0,10	[0, 1]	Coerencia da conviccao nas epocas recentes (<code>1 - std*5</code>)	Historico autorreferencial

Formula de conviccao:

```

indice_conviccao = 0,25 x (1 - entropia_crenca)
+ 0,25 x especializacao_gate
+ 0,20 x focus_conceito
+ 0,20 x acuracia_valor
+ 0,10 x estabilidade_papel

score_maturidade = EMA(indice_conviccao, alpha=0,3)

```

Thresholds de estado:

Estado	Condicao
DUVIDA	<code>conviction < 0,3</code>
CRISE	<code>conviction</code> cai > 20% do maximo movel dentro de 5 epocas
APRENDIZADO	<code>conviction</code> em <code>[0,3, 0,6]</code> e em aumento
CONVICTION	<code>conviction > 0,6</code> , estavel (<code>std < 0,05</code> em 10 epocas)
MATURE	<code>conviction > 0,75</code> , estavel por mais de 20 epocas, <code>value_accuracy > 0,7</code> , <code>gate_specialization > 0,5</code>

MaturitySnapshot **dataclass**: A cada epoca, o Observatorio produz um snapshot imutavel:

Campo	Tipo	Descricao
<code>epoch</code>	int	Numero da epoca
<code>timestamp</code>	datetime	Momento da gravacao
<code>belief_entropy</code>	float	Entropia de Shannon do vetor crenças
<code>gate_specialization</code>	float	Especializacao dos especialistas
<code>concept_focus</code>	float	Focalizacao nos conceitos de coaching
<code>value_accuracy</code>	float	Acuracia das predicoes de valor
<code>role_stability</code>	float	Estabilidade da classificacao dos papeis
<code>conviction_index</code>	float	Indice composto ponderado
<code>maturity_score</code>	float	Score EMA suavizado (alpha=0.3)
<code>state</code>	str	Estado atual (DUVIDA/CRISE/APRENDIZADO/CONVICCAO/MADURO)

Extracao dos 5 sinais neurais - como sao calculados:

Sinal	Metodo	Fonte de dados	Calculo
<code>belief_entropy</code>	<code>_compute_belief_entropy()</code>	<code>model._last_belief_batch</code>	softmax(belief) -> Shannon entropy / log(dim) -> 1 - normalizada
<code>gate_specialization</code>	<code>_compute_gate_specialization()</code>	<code>strategy.superposition.get_gate_statistics()</code>	1 - <code>mean_activation</code> (mais alto = especialistas mais especializados)
<code>concept_focus</code>	<code>_compute_concept_focus()</code>	<code>model.concept_embeddings.weight</code>	normas L2 -> softmax -> 1 - <code>entropy</code> (mais baixo = mais focalizado)
<code>value_accuracy</code>	<code>_compute_value_accuracy()</code>	Ciclo de validacao	1 - (<code>val_loss</code> / <code>initial_val_loss</code>), clamped [0, 1]
<code>role_stability</code>	<code>_compute_role_stability()</code>	Historico recente	1 - <code>std(ultimos 10 conviction_index)</code> x 5, clamped [0, 1]

API publica: `current_state` (propriedade -> string estado), `current_conviction` (propriedade -> float), `get_timeline()` (-> lista de `MaturitySnapshot` para exportacao/graficos).

Integracao TensorBoard: Cada `on_epoch_end()` registra **7 escalares** em TensorBoard:

```
maturity/belief_entropy, maturity/gate_specialization, maturity/concept_focus,
maturity/value_accuracy, maturity/role_stability,
maturity/conviction_index, maturity/maturity_score
```

Mais um log textual do estado atual via logger estruturado.

Garantias de design:

- **Impacto zero se desabilitado:** Quando nenhuma callback e registrada, todas as chamadas `CallbackRegistry.fire()` sao no-op. Nenhuma alocao de memoria, nenhum overhead de calculo.
- **Isolamento dos erros:** Cada callback e submetida individualmente a try/except-wrapping. Um erro de escrita de TensorBoard ou de calculo UMAP nunca causa o travamento do ciclo de training: o erro e registrado e o training continua.
- **Componivel:** E possivel adicionar novas callbacks subclassificando `TrainingCallback` e registrando-se com `CallbackRegistry.add()`. Nao e necessario modificar o codigo de training.

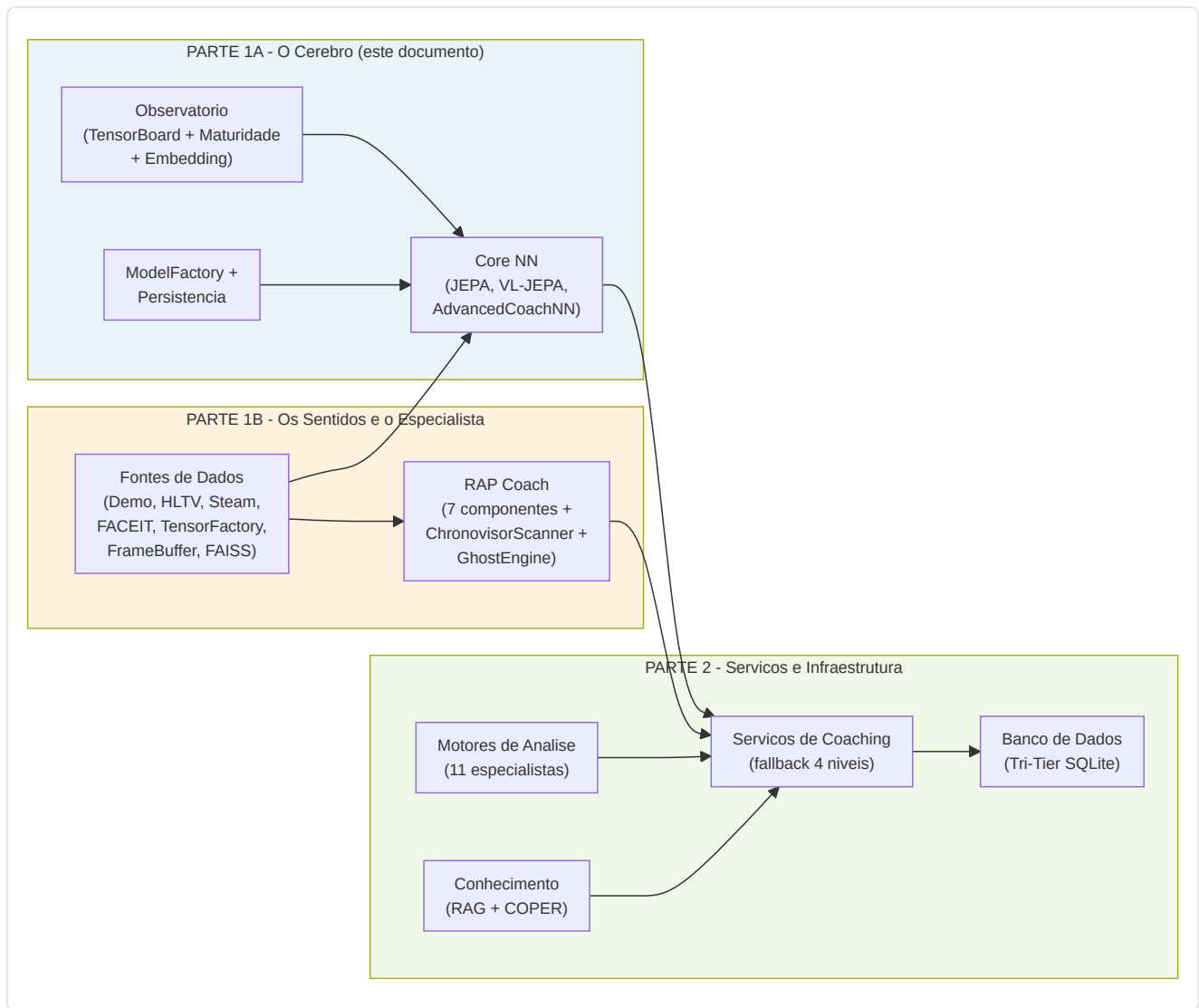
Integracao CLI: Iniciado atraves de `run_full_training_cycle.py` com os flags:

- `--no-tensorboard` - desabilita o callback de TensorBoard
- `--tb-logdir <caminho>` - define o diretorio de log de TensorBoard (predefinido: `runs/`)
- `--umap-interval <N>` - projecao UMAP a cada N epocas (predefinido: 10)

Resumo da Parte 1A - O Cerebro

A Parte 1A documentou o **nucleo cognitivo** do sistema de coaching - todo o subsistema de redes neurais que constitui o "cerebro" do coach:

Componente	Papel	Detalhes Chave
AdvancedCoachNN	Coaching supervisionado base	LSTM de 2 camadas + 3 especialistas MoE, input 25-dim, output 10-dim
JEPA	Pre-treinamento auto-supervisionado	Encoder online/target (EMA tau=0.996), InfoNCE contrastivo
VL-JEPA	Alinhamento visao-linguagem	16 conceitos de coaching em 5 dimensoes taticas
SuperpositionLayer	Gating contextual	Modulacao dependente do contexto 25-dim com observabilidade integrada
CoachTrainingManager	Orquestracao treinamento	3 niveis de maturidade (CALIBRACAO->APRENDIZADO->MADURO)
TrainingOrchestrator	Ciclo de epocas unificado	Early stopping, checkpoint, callback, pool negativos cross-match, quality gate
ModelFactory	Instanciacao modelos	6 tipos de modelo (+ RAP Lite) com persistencia e fallback
NeuralRoleHead	Classificacao papeis	MLP 5->32->16->5, consenso com heuristica
MaturityObservatory	Introspeccao treinamento	5 sinais -> indice de conviccao -> 5 estados de maturidade



Continua na Parte 1B - Os Sentidos e o Especialista: RAP Coach Model, ChronovisorScanner, GhostEngine, Fontes de Dados (Demo Parser, HLTV, Steam, FACEIT, TensorFactory, FrameBuffer, FAISS, Round Context)